

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Dahlquist, Julie, and Rainford Knight. *Principles of Finance*. OpenStax Press, 2022.

<https://assets.openstax.org/oscms-prodcms/media/documents/PrinciplesofFinance-WEB.pdf>.

- **Required pages:** Sections 15.1: Risk and Return to an Individual Asset; Section 15.2: Risk and Return to Multiple Assets (Printed pages 449-461; Ebook pages 463-475)
- **Estimated reading time:** 30 minutes

Lesson 1 - Portfolio Returns and Variance

December 25, 2025

MODULE 3 | LESSON 1

1 Portfolio Returns and Variance

Reading Time	2h
Prior Knowledge	python, returns, variance, diversification
Keywords	returns, logarithm, percent, matrix notation, distribution, annualize, geometric mean, arithmetic mean, portfolio variance, correlation, weights

In this lesson, we'll explore various types of returns, methods for annualizing returns, and techniques for calculating portfolio variance. The lesson aims to provide a comprehensive understanding of how to measure and interpret portfolio performance, considering both returns and risk.

We'll start by examining different types of returns, including percentage and logarithmic returns, and discuss their properties and appropriate use cases. We'll then move on to methods for annualizing returns, which is crucial for comparing investments over different time periods. The lesson will also cover portfolio variance calculation, exploring how individual asset variances, weights, and correlations contribute to overall portfolio risk.

1.1 1. Portfolio Returns

In Financial Markets, you were introduced to asset and portfolio returns. In Financial Data, we will explore both one-dimensional and multi-dimensional datasets. Therefore, matrix representations in linear algebra will be the main gateway for dealing with this subject. This makes sense because a portfolio consists of n assets with returns $r_i, i \in \{1, n\}$ that can be arranged in a matrix r where each column is represented by an asset. In this context, r_i are vectors, and **in the absence of perfect multicollinearity**, it is true that $\dim(\text{span}(r_i)) = n$.

We have already seen that the portfolio returns are:

$$r_p = \sum_1^n w_i \cdot r_i$$

where * r_p : portfolio returns * r_i : asset i returns * w_i : the weight of asset i

We should have an additional constraint in mind:

$$\sum_1^n w_i = 1$$

and for the purposes of this lesson, we will also assume that shorting is not allowed:

$$w_i > 0$$

Essentially, the return of the portfolio r_p is a linear combination of the returns of each asset. By construction, we have that $r_p \in \text{span}(r_i)$ for every combination of the weights. That means that the assets that an investor chooses are the basis (in a Linear Algebra context) of the portfolio.

Let's now write the portfolio returns in the format we will be using for the rest of the lesson and make the notation clear:

$$r_p = \mathbf{r} \cdot \mathbf{w}$$

with

$$\mathbf{r} = \begin{bmatrix} r_1 & r_2 & \dots & r_n \end{bmatrix}, \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

and each r_i is a vector that represents the returns of each asset across time t :

$$r_i = \begin{bmatrix} r_{t_1}^i \\ r_{t_2}^i \\ \vdots \\ r_{t_m}^i \end{bmatrix}$$

Using 1 as the sum of all weights has an intuitive explanation: we construct the portfolio by using a percentage of each asset (the weights) until we use the entire amount (1 means 100%). What does it mean if we break that constraint?

Exercise 1:

Use your favorite finance textbook and online resources in researching the constraint $\sum_1^n w_i = 1$. What would it mean for a portfolio if $\sum_1^n w_i = 2$? Answer the same question but now for $\sum_1^n w_i = 0$. Initiate a discussion in the forum explaining your understanding.

Let's download some real-world data:

```
[ ]: import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

```
[ ]: assets = ['MSFT', 'AAPL', 'AMZN', 'TSLA', 'GOOGL'] # Assets for portfolio
w = np.array([0.1, 0.2, 0.1, 0.4, 0.2]) # Weights of each asset

asset_prices = yf.download(assets, start='2018-01-01', end='2023-01-01',
    ↪auto_adjust = False)['Adj Close'] # Downloading daily data
asset_prices.index = pd.to_datetime(asset_prices.index) # Setting index as
    ↪datetime object

r = asset_prices.pct_change().dropna() # Calculating daily percent returns

r.head() # Each column is r_{i}

[ ]: r_port = r @ w # Creating portfolio returns
r_port.name = 'portfolio_returns'
r_port.head()
```

1.2 2 Types of Returns

In the example above, we have essentially presented a way to calculate portfolio daily percent returns. But these returns are not the only ones used in practice:

1.2.1 2.1 Percentage Returns

Percentage returns, also known as simple returns or arithmetic returns, are commonly used in practice:

$$r_t = \frac{p_t - p_{t-1}}{p_{t-1}}$$

where p_t is the price of an asset at time t .

The biggest bonus of using these returns is that the interpretation is intuitive: they directly represent the percentage change in value. A drawback is that they are not additive across time for one asset even though, for the exact same reason they are commonly used with portfolios.

The implications of nonadditivity are simple to show: imagine that you have an asset of price A . The first month, the price appreciates by 20%, but at the end of the second month, the price falls by 18%. If the returns were additive, one could just add them $20\% - 18\% = 2\%$ and argue that the total return for the 2 months is 2%. But as we will see in a future section, that is not true.

Lastly, the simple returns are not symmetrical. There is no boundary to the upside, yet it is bounded on the downside. Intuitively, the returns of a share cannot be less than -100% since the price would become negative. On the upside though, the returns can go really high especially when dealing with large horizons. Another way to showcase this is with the following observation: the price of an asset in three consecutive months is 100 -> 120 -> 100. At the end of the second month, the returns are 20%, but at the end of the third month, they are -16.6% . If the simple returns were symmetrical, then it would require the same absolute number of returns for going from 100 to 120 and then back to 100.

Exercise 2:

Create a custom function in Python that given a price array (for example `asset_prices['MSFT']`), will return the array of the arithmetic returns. You are not allowed to use any existing Python functions. Cross-check the results with the ones produced by using the `pct_change` pandas function.

1.2.2 2.2 Logarithmic Returns

The log returns, also known as continuously compounded returns, make use of the natural logarithm in the following way:

$$r_{t_{\log}} = \ln \left(\frac{p_t}{p_{t-1}} \right)$$

In contrast to the percent returns, the log returns are additive for one asset over time:

$$r_{(t_i - t_j)_{\log}} = \sum_{t=i}^{t=j} r_{t_{\log}}$$

they are also assumed (not always correctly) to be normally distributed, which is an important assumption for many econometric and stochastic models.

The additivity is responsible for the name “continuously compounded returns” since now you can just calculate the total returns over large horizons simply by adding the intra-period log returns. This creates a problem though: if we have the individual logarithmic returns of several assets, does it make sense to use them in order to calculate the returns of the portfolio? If adding log returns is a compounding process, are you allowed to add log returns of different assets?

The answer to the above questions is **no** and one needs to take extra care in order to use the correct type of return for each use case.

Lastly, the log returns are symmetrical. Mathematically, we can see that:

$$\log \left(\frac{p_t}{p_{t-1}} \right) = -\log \left(\frac{p_{t-1}}{p_t} \right)$$

By using the numbers of the above example, we can see that the log returns from 100 to 120 are equal (but negative sign) with the log returns of 120 to 100.

Exercise 3:

Create a custom function in Python that, given a price array (for example `asset_prices['MSFT']`), will return the array of the logarithmic returns.

As we have illustrated above, we can only distinguish between the two kinds of returns in terms of use cases. But now we need to be a bit more specific as to when and how. In order to do so, we need to do one observation:

```
[ ]: print("The 25th quantile is: ", r['MSFT'].quantile(0.25), "\n\nThe 75th quantile_
      ↪ is: ", r['MSFT'].quantile(0.75))
```

The above results illustrate that, on many occasions, the daily returns are of the order of 1%. Additionally, for the logarithmic and the percentage returns, it is true that:

$$r_{\log} = \ln\left(\frac{p_t}{p_{t-1}}\right) = \ln\left(\frac{p_t - p_{t-1} + p_{t-1}}{p_{t-1}}\right) = \ln(1 + r_{\text{pct}})$$

By using the Taylor expansion on the formula above, we get:

$$r_{\log} = \ln(1 + r_{\text{pct}}) = r_{\text{pct}} - \frac{r_{\text{pct}}^2}{2} + \frac{r_{\text{pct}}^3}{3} - \frac{r_{\text{pct}}^4}{4} + \dots$$

Since we observed that the daily returns are small < 0.01 , we can assume that the higher terms (r_{pct}^3 and above) will be very small:

$$r_{\log} = r_{\text{pct}} - \frac{r_{\text{pct}}^2}{2} + O(R^3)$$

By taking the expectation of both parts in the formula above and performing algebraic calculations, we have that:

$$\mathbb{E}(r_{\log}) \approx \mathbb{E}(r_{\text{pct}}) - \frac{\sigma^2}{2}$$

where σ is the variance of the percent returns.

The above result illustrates two important facts: 1. When volatility is low, we expect that the log returns and percent returns will be very close (Ruppert and Matteson). 2. When the sampling frequency gets lower, the log and percent returns tend to get larger in absolute values, and thus, volatility tends to get bigger as well. This results in divergence between the two returns.

```
[ ]: # Figure 1

pct_returns_msft = asset_prices['MSFT'].pct_change().dropna() # Calculating pct_
↪returns
log_returns_msft = np.log(asset_prices['MSFT'] / asset_prices['MSFT'].shift(1)).
↪dropna() # Calculating log returns

pct_change_msft_roll_mean = pct_returns_msft.rolling(15).mean() # Rolling_
↪average of pct returns
log_returns_msft_roll_mean = log_returns_msft.rolling(15).mean() # Rolling_
↪average of log returns

fig, ax = plt.subplots(figsize=(12,4))
ax.set_title('Difference between pct and log returns vs Price of MSFT')
ax.plot(asset_prices['MSFT'], label = 'MSFT Price', alpha = 0.7)
ax.set_ylabel('Price')
ax.set_xlabel('Date')
ax.legend()
```

```

ax2 = ax.twinx()
ax2.plot((pct_change_msft_roll_mean - log_returns_msft_roll_mean), color = 'red', alpha = 0.5, label = 'Difference between pct and log returns')
ax2.set_ylabel('Difference')
ax2.legend(loc = (0.01,0.89))
plt.show()

```

Figure 1 illustrates the difference between the pct and log returns. During low vol periods, both returns are very close, whereas during the March 2020 market crash, this difference becomes pronounced.

Exercise 4:

MSFT is a low-volatility stock. Recreate Figure 1 but now use TSLA in order to illustrate clearly in which periods the two returns diverge.

Exercise 5:

Argue why calculating the arithmetic weighted mean of log returns of different assets at a specific period in time will not give you the portfolio returns at that period.

Exercise 6:

Using the provided `asset_prices` DataFrame, calculate the daily portfolio value by computing the weighted sum of the asset prices, using predetermined asset weights (as you would when calculating portfolio returns). Next, calculate the logarithmic returns of this portfolio value series. Discuss the potential use cases and advantages of using log returns in this context.

Exercise 7:

Using the provided `asset_prices` DataFrame, calculate the cumulative return of each asset over the entire period using three different methods:

1. By using the daily percent returns array of each asset (hint: you will need to use the `prod` function).
2. By using the daily log returns.
3. By using the asset prices array.

1.3 3. Comparing Geometric and Arithmetic Returns

We will continue our discussion from Module 2 on how to annualize returns, but before delving into this subject, we will check a trivial example that illustrates the mean that should be used for each type of return, **if the compounding effect of returns is required**.

1.3.1 3.1 Geometric Mean Example

Let's assume that at time t_0 , you have 100 units. Between t_0 and t_1 , your return was 20%, and between t_1 and t_2 , it was -18%. We assume that t_i are spaced evenly.

Calculating the Arithmetic Mean

The arithmetic mean of the returns is:

$$r_{\text{mean}} = \frac{0.2 - 0.18}{2} = 0.01$$

This suggests that, on average, the return for each period is +1%. Following this logic, in the first period, we would have earned:

$$100 \cdot 0.01 = 1 \text{ unit}$$

So, at the end of the first period, we would have 101 units. During the second period, assuming the same +1% return:

$$101 \cdot 0.01 = 1.01 \text{ units}$$

This implies that at the end of the second period, we would have:

$$100 + 2.01 = 102.01 \text{ units in total.}$$

Using the Actual Returns

Now, let's follow a different approach by applying the actual returns sequentially.

1. At the end of t_1 , with a 20% return on 100 units:

$$100 \cdot 0.2 = 20 \text{ units earned.}$$

So, you have 120 units at the end of t_1 .

2. During the second period, from t_1 to t_2 , the return is -18%. Starting with 120 units:

$$120 \times 0.18 = 21.6 \text{ units lost.}$$

So, at the end of t_2 , you end up with:

$$120 - 21.6 = 98.4 \text{ units.}$$

Which Method Is Correct?

Let's calculate the geometric mean return to see which method aligns with reality:

$$r_{\text{GeomMean}} = \left(\prod_{i=1}^n (1 + r_i) \right)^{\frac{1}{n}} - 1$$

For our two periods:

$$r_{\text{GeomMean}} = ((1 + 0.2) \cdot (1 - 0.18))^{\frac{1}{2}} - 1 = \sqrt{1.2 \cdot 0.82} - 1 \approx 0.992 - 1 = -0.008$$

Now let's test this geometric mean to see if it reflects the actual ending value:

$$100 \cdot (1 - 0.008) \cdot (1 - 0.008) \approx 100 \cdot 0.992 \cdot 0.992 = 98.4 \text{ units.}$$

This example demonstrates the obvious: that the loss of 18% on 120 units is greater than the gain of 20% on 100 units, leading to an overall decrease in value. Therefore, despite the arithmetic mean suggesting a positive return of 1%, you actually lost money, as shown by the geometric mean. The geometric mean provides a more accurate representation of compounded returns over multiple periods, especially when returns are volatile.

That is of course when we deal with one asset across time and not when we deal with many assets at a specific point.

1.3.2 3.2 Arithmetic Mean

We have established that we cannot use the arithmetic mean with percent returns for one asset across time **if compounding is important for our analysis** (unless volatility is really low as we have explained). So the question is, can we use the arithmetic mean with the logarithmic returns? Let's rewrite the example above using the logarithmic returns instead of the simple ones.

The log returns are:

$$r_{\log_1} = \log(1.2) \approx 0.1823, \quad r_{\log_2} = \log(0.82) \approx -0.1987$$

Because log returns are additive, we can aggregate the returns as:

$$r_{\text{ArithMean}} = \frac{1}{2} (r_{\log_1} + r_{\log_2}) = \frac{1}{2} (\log(1.2) + \log(0.82))$$

By the properties of logarithms:

$$r_{\text{ArithMean}} = \frac{1}{2} \log(1.2 \cdot 0.82) \approx \log(0.984^{\frac{1}{2}}) \approx \log(0.992)$$

Exponentiating this gives the actual compounded return:

$$\exp(\log(0.992)) = 0.992$$

This corresponds to the geometric mean, which correctly reflects the final value:

$$100 \cdot 0.992 \cdot 0.992 \approx 98.4 \text{ units.}$$

Thus, the arithmetic mean $r_{\text{ArithMean}}$ of log returns provides the *correct* average return, aligning with the actual compounding of returns.

Exercise 8:

By using the properties of the logarithm, show that the log returns are indeed continuously compounded returns. (Hint: $r_{\log}^{t_1} + r_{\log}^{t_2} + r_{\log}^{t_3} + \dots = \ln\left(\frac{p_{t_2}}{p_{t_1}}\right) + \ln\left(\frac{p_{t_3}}{p_{t_2}}\right) + \dots$). Show that:

$$\ln(r_{\text{GeomMean}} + 1) = \frac{1}{n} \sum_{t=1}^{t=n} r_{\log}^t$$

Let's verify Exercise 8 using real-world data:

```
[ ]: percent_returns_tsla = asset_prices['TSLA'].loc["2020-06-01":"2020-09-30"].
    ↪ pct_change().dropna() # Calculating daily percent returns
geom_mean_tsla_1 = ((1 + percent_returns_tsla).prod() ** (1/
    ↪ len(percent_returns_tsla))) - 1 # Calculating geometric mean

log_returns_tsla = np.log(asset_prices['TSLA'].loc["2020-06-01":"2020-09-30"] /
    ↪ asset_prices['TSLA'].loc["2020-06-01":"2020-09-30"].shift(1)).dropna() #
    ↪ Calculating daily log returns
arithmetic_mean_tsla_1 = log_returns_tsla.mean() # Calculating arithmetic mean

print("Logarithm of geometric Mean of TSLA + 1: ", np.log(geom_mean_tsla_1 + 1))
print("Arithmetic Mean of TSLA: ", arithmetic_mean_tsla_1)
```

1.3.3 3.3 Choosing the Correct Mean for Percent Returns

When analyzing investment returns, it's essential to choose the appropriate mean based on the context. The arithmetic mean is **the correct measure for calculating expected returns in statistical models and portfolio optimization**. It represents the average return per period without considering the compounding effect. This is crucial when forecasting future returns, calculating expected portfolio returns, and performing risk assessments, as it aligns with the linear properties required in these calculations.

On the other hand, the geometric mean accurately reflects the average compounded growth rate over multiple periods. It accounts for the effects of compounding and is suitable for evaluating the historical performance of an investment over time. However, the geometric mean should not be used in place of the arithmetic mean when performing statistical analyses or modeling expected returns, as it does not provide an unbiased estimate of future performance.

In summary, while the geometric mean is invaluable for understanding the actual growth of an investment due to compounding, the arithmetic mean remains the fundamental measure in statistical contexts and portfolio management when assessing expected returns and making investment decisions.

1.4 4. Annualizing Percent Returns

The above (trivial in many cases) examples were used to create the intuition that you will need to distinguish between annualizing logarithmic and percent returns. Annualizing returns is crucial since it allows us to compare investments over different time periods on a common annual basis. This process is essential for evaluating performance and comparing different assets.

For percent returns, we use the geometric mean to account for compounding effects. The formula for annualizing daily percent returns is:

$$R_{\text{Annual}} = (1 + r_{\text{GeomMean}})^N - 1$$

Where:

- R_{Annual} is the annualized return
- N is the number of trading periods in a year (it could be 252 days or 12 months for stocks, or 365 days for continuously traded assets, etc.)

1.5 5. Annualizing Logarithmic Returns

For logarithmic returns, the annualization process is simpler due to their additive property. The formula for annualizing daily log returns is:

$$R_{\text{Annual,Log}} = N \cdot r_{\text{ArithMean}}$$

If we want to convert it back to the “actual” annual returns, then:

$$R_{\text{Annual}} = e^{N \cdot r_{\text{ArithMean}}} - 1$$

Let’s calculate the annual returns of MSFT.

```
[ ]: geom_mean_msft = ((1 + pct_returns_msft).prod() ** (1/len(pct_returns_msft))) -
    ↪ 1 # Calculating geometric mean
annual_pct_returns_msft = (1 + geom_mean_msft) ** 252 - 1 # Annualizing percent
    ↪ returns

annual_log_returns_msft = log_returns_msft.mean() * 252 # Annualizing log
    ↪ returns

print("Annualized pct returns of MSFT: ", annual_pct_returns_msft)
print("Annualized log returns of MSFT: ", annual_log_returns_msft)
```

Exercise 9:

Calculate the annual returns for TSLA, both for percent and log returns.

1.6 6. Portfolio Variance

1.6.1 6.1 Overview of Portfolio Variance

This was discussed at a high level during the Financial Markets course, but here, we will show how to calculate variance in Python with empirical data. While returns are important, investors are also concerned about risk or volatility.

But let’s think for a moment how we could measure risk for a portfolio of n assets. If we had one asset, then we could calculate the standard deviation of returns and this could give us an approximation of how the returns deviate from the mean return. Similarly, one can argue that we can calculate the portfolio returns and then the standard deviation.

```
[ ]: r_port.std() # Calculating standard deviation of portfolio returns
```

In Financial Markets, we also expressed portfolio variance as:

$$\sigma_p^2 = \sum_{i=1}^n \sum_{j=1}^n w_i w_j \text{Cov}(r_i, r_j)$$

Where:

- w_i = the portfolio weight of the i th asset
- $\text{Cov}_{i,j}$ = the covariance of two assets, which can be expressed as $\rho_{(i,j)} \sigma_i \sigma_j$, where $\rho_{(i,j)}$ is the correlation coefficient between the two assets

The formula below is equivalent to the one above, but it is given in matrix notation:

$$\sigma_p^2 = \mathbf{w}^T \cdot \Sigma \cdot \mathbf{w}$$

Where Σ is the portfolio covariance matrix.

```
[ ]: port_var = w.T @ r.cov() @ w # Calculating portfolio variance
port_std = np.sqrt(port_var) # Calculating portfolio standard deviation

print("Portfolio variance: ", port_var)
print("Portfolio standard deviation: ", port_std)
```

In the above cell, we have showcased that the intuitive and straightforward way of calculating portfolio variance (by computing std directly on the portfolio value) can be replicated by using the covariance matrix and the weights. What we have essentially done is decompose the portfolio variance into several components: * The weights * The individual variances * The between pairs covariances

The above result is somewhat anticipated. If the variance of an asset suddenly changes, the portfolio variance should change. If the weight of a high variance asset changes, then we expect that the portfolio variance will change accordingly. But what about covariances?

In order to see more clearly, we will decompose the covariance matrix a bit more:

$$\Sigma = D \cdot R \cdot D$$

where

- D is a diagonal matrix with the standard deviations of each asset
- R is the correlation matrix

The last decomposition shows us that the portfolio variance depends on the pairwise (Pearson) correlations as well.

1.6.2 6.2 Impact of Correlation on Portfolio Variance

Let's write the portfolio variance in its quadratic form but incorporating correlations instead of covariances:

$$\sigma_p^2 = \sum_{i=1}^n w_i^2 \sigma_i^2 + \sum_{i=1}^n \sum_{j \neq i}^n w_i w_j \sigma_{ij}$$

By substituting $\sigma_{ij} = \rho_{ij} \sigma_i \sigma_j$, we have:

$$\sigma_p^2 = \sum_{i=1}^n w_i^2 \sigma_i^2 + \sum_{i=1}^n \sum_{j \neq i}^n w_i w_j \rho_{ij} \sigma_i \sigma_j$$

To understand how the portfolio variance changes with respect to the correlation ρ_{ij} , we will take the partial derivative of σ_p^2 with respect to ρ_{ij} :

$$\frac{\partial \sigma_p^2}{\partial \rho_{ij}} = \frac{\partial}{\partial \rho_{ij}} \left(\sum_{i=1}^n w_i^2 \sigma_i^2 + \sum_{i=1}^n \sum_{j \neq i}^n w_i w_j \rho_{ij} \sigma_i \sigma_j \right)$$

The first sum is independent of ρ_{ij} , so its derivative with respect to ρ_{ij} is zero. The derivative of the second sum, with respect to ρ_{ij} , gives:

$$\frac{\partial \sigma_p^2}{\partial \rho_{ij}} = w_i w_j \sigma_i \sigma_j$$

We have already assumed that $w_i > 0$ and we know that standard deviations are always positive. That means that:

$$\frac{\partial \sigma_p^2}{\partial \rho_{ij}} > 0$$

Keeping all else the same, if the correlations between two assets increases/decreases, the portfolio variance will increase/decrease. Subsequently, if the correlations of many assets increase at the same time, the portfolio variance will increase faster, and that could be an indication of systemic risk.

Let's showcase some of the above facts with real-world data:

```
[ ]: # Figure 2
# We will visualize the asset's std and returns vs portfolio

fig, (ax1, ax2) = plt.subplots(1, 2, figsize = (14,4))

ax1.bar(x = r.columns, height = r.std(), alpha = 0.7)
ax1.hlines(y = port_std, xmin = -0.4, xmax = 4.4, linestyle = "--", color = 'red',
          alpha = 0.8, label = "Portfolio Std")
ax1.set_title("Assets vs Portfolio Volatility")
```

```

ax1.set_ylabel("Volatility")
ax1.legend()

ax2.bar(x = r.columns, height = (r + 1).prod() - 1, alpha = 0.7) # Make sure
    ↳you can explain why the `(r + 1).prod() - 1` is the total return
ax2.hlines(y = (r_port + 1).prod() - 1, xmin = -0.4, xmax = 4.4, linestyle =
    ↳"--", color = 'red', alpha = 0.8, label = "Portfolio Total Returns")
ax2.set_title("Assets vs Portfolio Total Returns")
ax2.set_ylabel("Total Returns (* 100%)")
ax2.legend()

plt.show()

```

In Figure 2, the weights we selected (randomly) gave us a portfolio whose volatility is close to that of the majority of the assets, but its returns are well above.

Let's test the relationship of the portfolio variance with the one of a random asset, for example MSFT.

```

[ ]: # Figure 3
multipliers = np.linspace(0.1,5,100) # Creating an array that will use to
    ↳multiply the MSFT returns array.
portfolio_variances = []
msft_variances = []

for multiplier in multipliers:
    temp_returns = r * np.array([1, 1, 1, multiplier, 1]) # Multiplying MSFT
    ↳returns with a number in order to scale the returns linearly, changing thus
    ↳the MSFT variance.
    temp_port_variance = w.T @ temp_returns.cov() @ w # Calculating portfolio
    ↳variance
    msft_variances.append((temp_returns['MSFT'] * multiplier).var())
    portfolio_variances.append(temp_port_variance)
    assert np.allclose(r.corr(), temp_returns.corr()) # For every new array of
    ↳returns where MSFT returns are multiplied with a number, the correlations
    ↳remain the same

fig, (ax1, ax2) = plt.subplots(1, 2, figsize = (16,5))

ax1.plot(multipliers, portfolio_variances)
ax1.scatter(1, port_var, color = 'red', label = "Initial Portfolio Variance
    ↳(non scaled MSFT returns)")
ax1.set_title("Portfolio Variance as MSFT variance is increasing")
ax1.set_xlabel("Multiplier")
ax1.set_ylabel("Variance")
ax1.legend()

```

```

ax2.plot(multipliers[10:30], portfolio_variances[10:30])
ax2.scatter(1, port_var, color = 'red', label = "Initial Portfolio Variance_
↳(non scaled MSFT returns)")
ax2.plot(multipliers[10:30], msft_variances[10:30], color = 'purple', label =_
↳"MSFT Variance (non scaled MSFT returns)")
ax2.set_title("Portfolio Variance vs MSFT variance")
ax2.set_xlabel("Multiplier")
ax2.set_ylabel("Variance")
ax2.legend()

plt.show()

```

Figure 3 illustrates the effects of the increase of the variance of one asset (MSFT in our case) on the variance of the portfolio. As we expect, the relationship of the portfolio variance and the variance of any asset is quadratic (left graph). The right graph illustrates the difference in risk of holding only one asset vs. a portfolio.

Let's now do the same but for correlations. For simplicity reasons, we will not simulate correlated assets (as you have already seen in Module 2 by using the Cholesky Decomposition), but we will merely create synthetic correlation matrices to show the effects of changing correlations in the portfolio variance.

In the next example, we keep variances and weights constant. We assume 100 assets for which we construct 18 correlation matrices. The 1st corr matrix is allowed to take values from -1 to 1. For each subsequent corr matrix, the low boundary changes as follows $-1 + 0.1 \cdot j$. That means that the 2nd corr matrix will have correlations from -0.9 to 1 and the 18th corr matrix from 0.8 to 1.

```

[ ]: n_assets = 100

w = np.random.dirichlet(np.ones(n_assets), size=1)[0] # Simulate weights using_
↳Dirichlet distribution

degrees_of_freedom = 8 # Adjust the skewness of the Chi-square distribution
variances = np.random.chisquare(df=degrees_of_freedom, size=n_assets)
variances = variances / np.max(variances) * 0.003 # Scale to max variance of 0.
↳003
std_devs = np.sqrt(variances)
D = np.diag(std_devs)

# Construct the correlation matrices
corr_matrices = []
for j in range(19):
    R = np.zeros((n_assets, n_assets))
    low_boundary = -1 + j*0.1
    for i in range(1, n_assets):
        R[i, :i] = np.random.uniform(low_boundary, 1, i) # Creating the lower_
↳triangular part of the correlation matrix

```

```

R = R + R.T # Comment this line yourself
np.fill_diagonal(R, 1)
corr_matrices.append(R)

# Calculate portfolio variances
portfolio_variances = []
for R in corr_matrices:
    portfolio_variance = w.T @ D @ R @ D @ w
    portfolio_variances.append(portfolio_variance)

# Plot portfolio variance against correlations
x_labels = [str((round(-1 + j*0.1, 1), 1)) for j in range(19)]
plt.figure(figsize = (10,5))
plt.plot(portfolio_variances)
plt.title("Portfolio Variance vs Correlation")
plt.xlabel("Correlation Interval")
plt.ylabel("Variance")
plt.xticks(ticks = range(19), labels = x_labels, rotation = 45)
plt.show()

```

Students who do not have a finance background may not have expected that result. Even when the individual variances and the weights remain constant, changes in correlations affect the portfolio variance. Specifically in our example, the portfolio variance is the lowest when the correlations were sampled from uniform distribution and interval $[-1, 1]$. As the assets became more correlated, the variance went up as well.

Exercise 10:

Select $n \geq 2$ stocks/bonds and construct a portfolio (select appropriate weights) using historical data such that the portfolio variance is 0 (or really close to 0 in order to be considered 0) and the returns are positive (far from 0). Present your findings in the forum. If you did not find one, present the reasons you could not find one.

1.7 7. Portfolio Weights

Assume we can construct a portfolio that consists of stocks, bonds, and crypto. An investor might choose to hold a lot of bonds while another chooses mostly stocks and a third one loves crypto. The choice has to do with the risk profile of the investor; in other words, the choice of weights will satisfy the needs for specific returns given a risk tolerance (variance).

From this perspective, the choice of weights is a subjective matter, and indeed in practice, investors behave very differently (Elton et al.). One of the best ways to explore this is by constructing the Efficient Frontier graph.

```

[ ]: from scipy.optimize import minimize

weights = np.random.dirichlet(np.ones(5)*0.7, size = 20000) # Creating 20000
↳ sets of weights using dirichlet distribution

```

```

assert np.isclose(np.sum(weights, axis = 1), 1).all() # Check that each set of
↳weights sum up to 1

eff_front_dict = {}
cov_matrix_ret = r.cov() * 252
expected_returns = r.mean() * 252

# Filling the eff_front_dict
for w in weights:
    port_ret = expected_returns @ w.T # Annualized percent returns as expected
↳returns
    port_std = np.sqrt(w.T @ cov_matrix_ret @ w)
    eff_front_dict[str(list(w))] = [port_ret, port_std]

eff_frontier_dataframe = pd.DataFrame(eff_front_dict, index = ['Returns',
↳'Standard Deviation']).T # Storing everything in one dataframe

def get_portfolio_stats(weights, expected_returns, cov_matrix):
    port_ret = expected_returns @ weights
    port_std = np.sqrt(weights.T @ cov_matrix @ weights)
    return port_ret, port_std

def negative_sharpe_ratio(weights, expected_returns, cov_matrix,
↳risk_free_rate=0.02):
    port_ret, port_std = get_portfolio_stats(weights, expected_returns,
↳cov_matrix)
    sharpe_ratio = (port_ret - risk_free_rate) / port_std
    return -sharpe_ratio

def minimum_variance(weights, expected_returns, cov_matrix):
    return get_portfolio_stats(weights, expected_returns, cov_matrix)[1]

def efficient_frontier_point(expected_returns, cov_matrix, target_return):
    n_assets = len(expected_returns)
    args = (expected_returns, cov_matrix)
    constraints = (
        {'type': 'eq', 'fun': lambda w: np.sum(w) - 1},
        {'type': 'eq', 'fun': lambda w: get_portfolio_stats(w,
↳expected_returns, cov_matrix)[0] - target_return}
    )
    bounds = tuple((0, 1) for _ in range(n_assets))

    result = minimize(
        minimum_variance,
        x0=np.ones(n_assets) / n_assets,
        args=args,

```

```

        method='SLSQP',
        bounds=bounds,
        constraints=constraints
    )

    return result.x

def get_efficient_frontier(expected_returns, cov_matrix, n_points=100):
    # Find the minimum variance portfolio
    n_assets = len(expected_returns)
    args = (expected_returns, cov_matrix)
    constraints = ({'type': 'eq', 'fun': lambda w: np.sum(w) - 1})
    bounds = tuple((0, 1) for _ in range(n_assets))

    min_var_result = minimize(
        minimum_variance,
        x0=np.ones(n_assets) / n_assets,
        args=args,
        method='SLSQP',
        bounds=bounds,
        constraints=constraints
    )
    min_ret, min_std = get_portfolio_stats(min_var_result.x, expected_returns, cov_matrix)

    # Find the maximum return portfolio
    max_ret_idx = np.argmax(expected_returns)
    max_ret = expected_returns[max_ret_idx]

    # Generate points on the efficient frontier
    target_returns = np.linspace(min_ret, max_ret, n_points)
    efficient_portfolios = []

    for target_return in target_returns:
        weights = efficient_frontier_point(expected_returns, cov_matrix, target_return)
        ret, std = get_portfolio_stats(weights, expected_returns, cov_matrix)
        efficient_portfolios.append([std, ret])

    return np.array(efficient_portfolios)

# Calculate the efficient frontier points
efficient_points = get_efficient_frontier(expected_returns, cov_matrix_ret)

# Plot
plt.figure(figsize=(10,6))
plt.scatter(x=eff_frontier_dataframe['Standard Deviation'],

```

```

        y=eff_frontier_dataframe['Returns'],
        alpha=0.4)
plt.scatter(x=r.std() * np.sqrt(252),
            y=expected_returns,
            color='red',
            label="Individual Assets",
            alpha=0.7)
plt.plot(efficient_points[:,0],
        efficient_points[:,1],
        'orange',
        linewidth=3,
        label='Efficient Frontier',
        alpha=0.6)
plt.title("Efficient Frontier")
plt.xlabel("Annual Portfolio Standard Deviation")
plt.ylabel("Annual Portfolio Returns")
plt.legend()
plt.grid(True)
plt.show()

```

Let's read the above illustration, which admittedly is one of the most important in portfolio management.

First of all, we need to observe the shape of the frontier: the curve-like line that spans the left and upper part of the scatter plot represents all possible optimal portfolios that offer the highest expected return for a given level of risk (or the lowest risk for a given level of expected return). This curve is called the **efficient frontier**.

Secondly, if we were to draw vertical lines on this graph, each line would intersect the frontier at a point representing the portfolio with the highest return for that specific level of risk. At the same time, there are plenty of weights that expose the portfolio to the same level of risk but for less return. That should bring our attention to the fact that even though the weights are subjective, if the portfolio that an investor chose does not lie in the efficient frontier, then the investor can do better in terms of returns by following the vertical line upwards until reaching the efficient frontier. In short, for any given level of risk (variance), there is an optimal portfolio composition that maximizes returns.

The same as above is true if we were to draw horizontal lines. In this case, we can claim that for any given level of returns, we can minimize risk by following the horizontal line to the left up until it crosses the efficient frontier.

As we move up and to the right along the frontier, we see higher potential returns but also higher risk. This illustrates the fundamental principle that to achieve higher returns, one generally must accept higher risk. The concave shape of the frontier illustrates the benefits of diversification. Portfolios on the frontier are typically well diversified, offering better risk-return profiles than individual assets.

Exercise 11:

Create your own portfolio, one to your liking: choose one or more equities, ETFs, bonds, and crypto. Try to include assets that are not that correlated with the rest of the portfolio (if this is

possible) and also include some very high-volatility assets (large marketcap altcoins or newly issued ones). Use the code provided above in order to construct the efficient frontier. Paste the graph in the forums along with a **small** paragraph that lists the assets chosen.

Exercise 12 (Optional):

In the first paragraph of this notebook, we mentioned that given n vectors r_i , in the absence of multicollinearity, it is true that $\dim(\text{span}(r_i)) = n$. Otherwise, the dimension of the span will be lower than n . This is a known result from linear algebra. Use your favorite linear algebra textbook in order to remember what multicollinearity is and then argue why an investor would keep an asset in their portfolio when that asset could be written as a perfect linear combination of the rest of the portfolio assets. Should the investor just omit such an asset? What is its contribution to lowering portfolio variance or increasing returns? Does it give flexibility in portfolio management?

Exercise 13:

Throughout this lesson, students were introduced to the correct usage of the arithmetic and geometric mean. In Section 2.4 *Annualizing Returns*, we explained how to annualize the percent and log returns if the compounding effect was needed. Still, when constructing the efficient frontier, we did this: `expected_returns = r.mean() * 252`. Why? In your answer, consider the context of portfolio optimization and whether we are focusing on modeling expected returns for statistical purposes or calculating the actual historical growth of an investment due to compounding. Discuss the implications of using the arithmetic mean in this context.

1.8 8. Conclusion

In this lesson, we learned that logarithmic returns are additive and useful for longer-term analysis, while percentage returns are more intuitive for short-term performance evaluation. We also discovered how to annualize returns, allowing for fair comparisons between investments with different time horizons. The lesson emphasized the importance of understanding portfolio variance and its components. We saw how individual asset variances, weights, and correlations all play crucial roles in determining overall portfolio risk. The efficient frontier concept illustrated the trade-off between risk and return, highlighting the benefits of diversification.

Through practical examples and exercises using Python, we've gained hands-on experience in calculating and interpreting these metrics. This knowledge forms a crucial foundation for more advanced topics in portfolio management and financial analysis.

References

- Elton, Edwin J., et al. *Modern Portfolio Theory and Investment Analysis*. John Wiley & Sons, 2009.
- Ruppert, David, and David S. Matteson. *Statistics and Data Analysis for Financial Engineering*. 2nd ed., Springer, 2011.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

1. Dahlquist, Julie, and Rainford Knight. *Principles of Finance*. OpenStax Press, 2022. (open access at <https://assets.openstax.org/oscms-prodcms/media/documents/PrinciplesofFinance-WEB.pdf>).
 - **Required pages:** Section 15.3: The Capital Asset Pricing Model and Section 15.4: Applications in Performance Measurement (printed pp. 461-467; ebook pp. 475-481)
 - **Estimated reading time:** 20 minutes
2. Dowd, Kevin, and David Blake. "After VaR: The Theory, Estimation, and Insurance Applications of Quantile-Based Risk Measures." *Journal of Risk and Insurance*, vol. 73, no. 2, 2006, pp. 193-229.
<https://www.nottingham.ac.uk/business/who-we-are/centres-and-institutes/gcbfi/documents/cris-reports/cris-paper-2006-2.pdf#page=3.12>.
 - **Required pages:** Sections 2.1-2.4 and 3.1-3.3
 - **Estimated reading time:** 40 minutes

Lesson 2 - Efficient Frontier, Sharpe Ratio and Downside Risk Metrics

December 25, 2025

MODULE 3 | LESSON 2

1 Efficient Frontier, Sharpe Ratio and Downside Risk Metrics

Reading Time	6h
Prior Knowledge	percent-log returns, risk metrics, normal and t distributions
Keywords	dividend adjusted, adjusted close, splits, returns, Value at Risk, Conditional, expectation

In this lesson, we continue by exploring some important types of returns. Specifically, we introduce total returns, dividend-adjusted returns, and excess returns. We discuss the adjusted close price and give direction on how to calculate it. We then progress to the Sharpe Ratio, a fundamental metric in modern portfolio theory that measures risk-adjusted performance. This ratio helps investors and analysts evaluate the return of an investment in relation to its risk. Lastly, we introduce the Value at Risk (VaR), a key risk metric used widely in the financial industry. We cover multiple methods for calculating VaR, including historical, parametric (using both normal and t-distributions), and Monte Carlo simulation approaches. We also introduce the concept of Conditional Value at Risk (CVaR), which provides additional insight into tail risk.

```
[ ]: !pip install --upgrade pip
      !pip install yfinance --upgrade
```

```
[ ]: import pandas as pd
      import numpy as np
      import matplotlib.pyplot as plt
      import seaborn as sns
      import yfinance as yf
      import fredapi as fa
      from scipy import stats
      import numpy as np
      sns.set()
```

1.1 1. Total Returns and Dividend-Adjusted Returns

In the previous lesson, we discussed on portfolio returns and variance. We specifically learned how to calculate returns and annualize them along with some specific characteristics of the two major kinds of returns: percent and logarithmic.

In this lesson, we start by expanding on **total, dividend-adjusted, and excess Returns** as a prerequisite for various kinds of ratios an engineer should be aware of. In doing so, a discussion on *adjusted close price* will naturally emerge.

1.1.1 1.1 Total Returns & Dividend Adjusted Returns

Total returns represent the overall gain or loss on an investment over a specific period. It accounts for all forms of income:

$$\text{Total Returns} = \frac{(p_T - p_0) + (D + I)}{p_0}$$

Where: * T : End of period. * D : Dividends. * I : Any form of additional income the specific investment provides.

Dividend-adjusted returns are similar to total returns except they only account for dividends paid:

$$\text{Dividend Adjusted Returns} = \frac{(p_T - p_0) + D}{p_0}$$

The above two formulae are self-explanatory and intuitive, but in order to dive a bit deeper, we will need to calculate them given a close price and the dividends per share.

Let's consider a stock X with the following close price and dividends over three years (all prices are ex-date):

Year	Price (\$)	Dividends(\$)
2019:	105	2
2020:	112	2.2
2021:	118	2.4
2022:	116	2.3

The dividend-adjusted returns for the period 2021 to 2022 are:

$$r_{2021-2022}^{\text{Adj}} = \frac{116 - 118 + 2.3}{118} = 0.0025$$

whereas the non-adjusted returns are:

$$r_{2021-2022} = \frac{116 - 118}{118} = -0.017$$

Even though the two returns are pretty close, their sign is different! And since every investor that holds shares will get paid dividends, we need to take this into account as well in calculating

returns. The latter leads us to the adjusted-close price as a convenient price transformation that takes dividends into account (along with splits and other corporate actions)

1.1.2 1.2 Adjusted-Close Price

A simple way of calculating the adjusted-close (assuming no splits) is the following:

$$p_t^{\text{adj}} = p_t^{\text{close}} - \sum_{i>t} D_i$$

which essentially means that at any given time t , we need to subtract all dividends that are paid from time t and on. Calculating the adjusted-close is a backwards-looking procedure. For example:

Year	Close Price (\$)	Dividends(\$)	Adjusted Close (\$)
2019:	105	2	$105 - 2.2 - 2.3 - 2.4 = 98.10$
2020:	112	2.2	$112 - 2.3 - 2.4 = 107.30$
2021:	118	2.4	$118 - 2.3 = 115.07$
2022:	116	2.3	116

So now, let's calculate the dividend-adjusted returns and the adjusted-close returns over the period of 2019 to 2021.

$$\text{Dividend Adjusted Returns} = \frac{(118 - 105) + 2.4 + 2.2}{105} = 0.1676$$

and

$$\text{Adjusted Close Returns} = \frac{115.07 - 98.10}{98.10} = 0.1723$$

The above two numbers are different. Do you know why?

The reason lies in the reinvestment assumption when calculating the adjusted-close price. One can see that this transformation leads to significant changes in the price array: for 2019, the observed price was 105 whereas the adjusted-close was 98.10. Similarly, for 2021, the observed price was 118 while the adjusted-close was 115.07. The difference between the two adjusted-close prices is attributed to price appreciation (the same as with the close prices) and the dividends paid during that period. In essence, the share's close price along with the dividends **is replicated with a price that incorporates the dividends as if the latter was a part of price appreciation.**

But if the price of an asset one holds appreciates, then the appreciation is still part of the asset and not cash as is the case when dividends are paid. In this observation, we find an important distinction between the dividend-adjusted returns and adjusted-close returns: when using adjusted-close returns, we assume that we reinvest the dividends in more shares.

For a more detailed and precise way of calculating the adjusted close, please read this: <https://help.yahoo.com/kb/SLN28256.html?guccounter=1>. Make sure you can explain the benefits of using multipliers instead of subtracting the dividends in adjusting the price.

Exercise 1

Given an additional data point for stock X:

Year	Price (\$)	Dividends(\$)
2023:	120	1.8

answer whether the close price for 2020 will change. What about the existing adjusted close price for 2020? By how much? (Use the method provided in the notebook.)

Now, let's showcase an example of calculating the adjusted-close price for TSLA, which had two splits but never paid dividends (make sure you understand the method explained in the link above).

```
[ ]: # Figure 1
tsla = pd.read_csv("https://raw.githubusercontent.com/DonLeKouT/Just_files/refs/heads/main/TSLA_stock_price.csv")
tsla.set_index('date', inplace=True)
tsla.index = pd.to_datetime(tsla.index)

splits = {'2022-08-25' : 3, "2020-08-31" : 5} # The dates and multiplier of TSLA splits

tsla['Adjusted Close'] = tsla['close'].copy()

for date, split in splits.items():
    date = pd.to_datetime(date) - pd.Timedelta(days=1)
    tsla.loc[:date, 'Adjusted Close'] = tsla.loc[:date, 'Adjusted Close'] / split

tsla[['close', 'Adjusted Close']].loc["2016":].plot(figsize=(12, 8), subplots = True)
plt.show()
```

In Figure 1, one can see the differences between the close and the adjusted-close price. We notice the divergence in scale (y -axis), which is proportional to the splits. Additionally, in the close price plot, we observe the vertical decline in price on the days the split happened. That is to be expected since the split kept the market cap of TSLA intact, lowering the price of the share in an instant.

Exercise 2

Compute the daily returns (percent or log) of the close and adjusted-close price of TSLA. Are they different?

Exercise 3

Create an account in AlphaVantage and obtain an APIKEY. Select an asset that had undergone several splits (2 or more) and paid some dividends (twice or more) and download the close price (not adjusted). Use the Yahoo methodology, found in the link provided above, in order to calculate the adjusted-close price. Plot the results as in Figure 1.

1.2 2. Excess Returns

1.2.1 2.1 Excess Returns Definition

As you already remember from the Financial Markets course, excess return is the difference between the returns of an asset (or portfolio) and a benchmark. An analyst can use various benchmarks depending on the analysis, but the most common ones are a market proxy (S&P500) or a risk-free rate (T-Bills). The formula for excess returns is:

$$\text{Excess Returns} = r_{\text{portfolio}} - r_{\text{benchmark}}$$

where: * $r_{\text{portfolio}}$ is the **total return** of the investment * $r_{\text{benchmark}}$ is the returns of the benchmark

In calculating excess returns, it is crucial to account for all types of cash flows associated with the investment, which is why total returns are used. In practice, however, adjusted-close returns can be used, provided that any additional cash flows are manually included if they are not already factored into the adjusted-close price by the data provider.

But now, we need to talk about the benchmark. As we have mentioned, the benchmark is case specific and can be represented by any asset an analyst decides to test the returns of their investment against. For example:

- Selecting a benchmark in terms of objectives, strategy, and sector: S&P500 as a broad market benchmark, NASDAQ for technology and growth portfolios, or Russell 2000 for small-cap equity portfolios.
- Geographic exposure: the benchmark should represent the region where the investment takes place. In the UK, one could use FTSE 100 or FTSE 250; in the EU, there is EURO STOXX 50, DAX, or CAC 40; and in India, one has NIFTY 50, S&P BSE 500, and more.
- Using the risk-free rate as the benchmark, one should take into account the:
 - Investment horizon: for short-term investments, government securities like 3-month T-bill are appropriate. For longer-term investments, 10-year treasuries could be more suitable. In any case, the rule of thumb is to use the rate that is the closest to the duration of the investment.
 - Theoretical considerations: the 3-month T-bill is widely used in academic studies and models like CAPM, but the 10-year Treasury better represents the expectations for inflation and the economy.

1.2.2 2.2 Excess Returns over Expected Return

In the context of finance, the expected return of an asset represents the return an investor expects/requires given the risk profile of the specific asset. That means that in calculating the

expected return, one can take into account the systematic risk, inflation expectations, interest rate risk, and more. If the actual returns of the asset are greater than expected returns, then we can say that *the asset has outperformed expectations*.

$$\text{Excess Returns} = r_{\text{actual}} - \mathbb{E}(r_{\text{asset}})$$

But calculating the expected return is not straightforward and depends on several strong assumptions. We will mention several methods used to estimate the expected returns of an asset. These methods are distinguished with respect to the types of risk that they make use of:

CAPM

A widely used model is the capital asset pricing model (CAPM), which models returns with respect to systematic risk. The formula for CAPM is:

$$\mathbb{E}(r_{\text{asset}}) = r_f + \beta(r_m - r_f)$$

where * r_f is the risk-free rate * r_m is a market proxy * β is the sensitivity of the asset to the market returns.

In the CAPM world, the excess return is often called *alpha* (α):

$$\alpha = r_{\text{actual}} - (r_f + \beta(r_m - r_f))$$

APT

The arbitrage pricing theory (APT) goes beyond the CAPM model in incorporating multiple factors of risk instead of just the systematic one (market). These factors could be any macroeconomic variable such as inflation, interest rates, economic growth, etc. The formulation is:

$$\mathbb{E}(r_{\text{asset}}) = r_f + \beta_1 f_1 + \beta_2 f_2 + \dots + \beta_n f_n$$

where * f_i are the factor risk premiums * β_i are the sensitivities of the asset to these factors.

Fama-French Three-Factor Model

This model extends CAPM by adding two additional factors: 1. The *SMB* (small minus big) factor, which represents the “size premium.” This is essentially the difference between small-cap and large-cap stocks. 2. The *HML* (high minus low) factor, which measures the difference between the returns of the value and growth stocks.

As with the CAPM and APT models, Fama-French also assumes linearity between expected returns and factors:

$$\mathbb{E}(r_{\text{asset}}) = r_f + \beta_{\text{market}}(r_m - r_f) + \beta_{\text{size}} \cdot \text{SMB} + \beta_{\text{value}} \cdot \text{HML}$$

The three models above were merely mentioned to explain the notion of expected returns and its importance in calculating excess returns. Essentially, excess returns measure performance over a chosen benchmark, which can be an external index or the asset’s own expected return based on models like CAPM or APT.

1.3 3. Portfolio Sharpe Ratio and the Efficient Frontier

We have already mentioned what the Sharpe ratio is in the previous module. We will now illustrate the importance of the Sharpe ratio in the context of portfolio management. In order to do so, we will assume the assets of Lesson 1, and we will show where the highest Sharpe ratio portfolios are located with respect to the efficient frontier.

```
[ ]: assets = ['MSFT', 'AAPL', 'AMZN', 'TSLA', 'GOOGL'] # Assets for portfolio

asset_prices = yf.download(assets, start='2018-01-01', end='2023-01-01',
    ↪auto_adjust = False)['Adj Close'] # Downloading daily data
asset_prices.index = pd.to_datetime(asset_prices.index.date)

# Download risk-free rate data (3-month Treasury bill)
risk_free = yf.download('^IRX', start='2018-01-01', end='2023-01-01',
    ↪auto_adjust = False)['Adj Close']
risk_free.index = pd.to_datetime(risk_free.index.date)

r = asset_prices.pct_change().dropna() # Calculating daily percent returns

[ ]: # Figure 2
weights = np.random.dirichlet(np.ones(5)*0.7, size = 20000) # Creating 20000
    ↪sets of weights using dirichlet distribution

assert np.isclose(np.sum(weights, axis = 1), 1).all() # Check that each set of
    ↪weights sum up to 1

eff_front_dict = {}
cov_matrix_ret = r.cov() * 252
expected_returns = r.mean() * 252
risk_free_rate = risk_free['^IRX'].mean() / 100 # The rates are already
    ↪annualized

# Filling the eff_front_dict
for w in weights:
    port_ret = expected_returns @ w.T # Annualized percent returns as expected
    ↪returns
    port_std = np.sqrt(w.T @ cov_matrix_ret @ w)
    sharpe_ratio = (port_ret - risk_free_rate) / port_std
    eff_front_dict[str(list(w))] = [port_ret, port_std, sharpe_ratio]

eff_frontier_dataframe = pd.DataFrame(eff_front_dict, index = ['Returns',
    ↪'Standard Deviation', 'Sharpe Ratio']).T # Storing everything in one
    ↪dataframe
highest_sharpe_ratio = eff_frontier_dataframe.sort_values(by = 'Sharpe Ratio',
    ↪ascending = False) # Sorting the portfolios with respect to Sharpe Ratio
```

```

# Plot the portfolio returns vs portfolio variance
plt.figure(figsize = (10,6))
plt.scatter(x = eff_frontier_dataframe['Standard Deviation'], y =
    ↪eff_frontier_dataframe['Returns'], alpha = 0.4)
plt.scatter(x = r.std() * np.sqrt(252), y = expected_returns, color = 'red',
    ↪label = "Individual Assets", alpha = 0.7)
plt.scatter(x = highest_sharpe_ratio[:5000]['Standard Deviation'], y =
    ↪highest_sharpe_ratio[:5000]['Returns'], color = 'purple', alpha = 0.2, label =
    ↪"The 5000 portfolios with the best Sharpe Ratio" )
plt.scatter(x = highest_sharpe_ratio[:1000]['Standard Deviation'], y =
    ↪highest_sharpe_ratio[:1000]['Returns'], color = 'yellow', alpha = 0.3, label =
    ↪"The 1000 portfolios with the best Sharpe Ratio" )
plt.scatter(x = highest_sharpe_ratio[:100]['Standard Deviation'], y =
    ↪highest_sharpe_ratio[:100]['Returns'], color = 'green', alpha = 0.3, label =
    ↪"The 100 portfolios with the best Sharpe Ratio" )
plt.scatter(x = highest_sharpe_ratio.iloc[0,:]['Standard Deviation'], y =
    ↪highest_sharpe_ratio.iloc[0,:]['Returns'], color = 'black', alpha = 0.8,
    ↪label = "Best Sharpe Ratio portfolio" )

plt.title("Efficient Frontier")
plt.xlabel("Annual Portfolio Standard Deviation")
plt.ylabel("Annual Portfolio Returns")
plt.legend()
plt.show()

```

What we essentially see in Figure 2 is the efficient frontier, as explained in the previous lesson, onto which we have additionally plotted the highest Sharpe Ratio portfolios.

More specifically, we illustrate where the 5000, 1000, and 100 portfolios with the highest Sharpe ratios are located and of course where the best Sharpe ratio portfolio is. As expected (?), those portfolios are on or very close to the efficient frontier, but to our surprise, they do not occupy the entire efficient frontier curve.

Exercise 4

Explain why we expect that the best Sharpe ratio portfolio will be on the efficient frontier.

Exercise 5 (for discussion)

Use the `highest_sharpe_ratio` dataframe and print the 10 portfolios with the highest Sharpe ratio. If you look at the weights closely, you will observe that while the Sharpe ratios between these portfolios are very similar, the weights vary a lot. For example, the weight for the 1st asset in the 8th portfolio is 44.8%, but the weight for the same asset in the 7th portfolio is 18.7%. However, the difference in their Sharpe ratios is minimal. Think it over and then discuss with your peers in the forum the implications of the above observation on portfolio management: that several very different portfolios have almost identical Sharpe ratios.

1.4 4. The Tangency Portfolio and the Tangent Line

Given the above observations, we can now give a definition for the best Sharpe ratio portfolio: the best Sharpe ratio portfolio, or the tangency portfolio, is the one that maximizes returns per unit of risk. **It provides the best risk-return tradeoff among all portfolios** (Elton et al.).

Our discussion and illustrations of portfolios' efficient frontiers give the impression that the efficient frontier contains optimal portfolios. In the next section, we will discuss the capital market line in order to illustrate how the tangency portfolio along with the risk-free rate can be used in creating a series of portfolios that are superior to the efficient frontier.

The tangent line, or the capital market line, is a key concept that arises from combining a risk-free asset with the tangency portfolio. Mathematically, the CML formulation is:

$$r_{\text{CML}} = r_f + \frac{r_T - r_f}{\sigma_T} \cdot \sigma_P$$

where * r_{CML} is the expected return of any portfolio on the CML * r_T is the return of the tangency portfolio (best Sharpe ratio portfolio) * σ_T is the std of the tangency portfolio * σ_P is the std of any portfolio on the CML

Let's visualize the CML before providing some intuition behind its use:

```
[ ]: sigmas = np.linspace(0,0.7, 100)
CML = risk_free_rate + highest_sharpe_ratio['Sharpe Ratio'].iloc[0] * sigmas #
↳ This is the Capital Market Line

# Plot the portfolio returns vs portfolio variance
plt.figure(figsize = (14,8))
plt.scatter(x = eff_frontier_dataframe['Standard Deviation'], y =
↳ eff_frontier_dataframe['Returns'], alpha = 0.4)
plt.scatter(x = r.std() * np.sqrt(252), y = expected_returns, color = 'red',
↳ label = "Individual Assets", alpha = 0.7)
plt.scatter(x = highest_sharpe_ratio[:5000]['Standard Deviation'], y =
↳ highest_sharpe_ratio[:5000]['Returns'], color = 'purple', alpha = 0.2, label
↳ = "The 5000 portfolios with the best Sharpe Ratio" )
plt.scatter(x = highest_sharpe_ratio[:1000]['Standard Deviation'], y =
↳ highest_sharpe_ratio[:1000]['Returns'], color = 'yellow', alpha = 0.3, label
↳ = "The 1000 portfolios with the best Sharpe Ratio" )
plt.scatter(x = highest_sharpe_ratio[:100]['Standard Deviation'], y =
↳ highest_sharpe_ratio[:100]['Returns'], color = 'green', alpha = 0.3, label =
↳ "The 100 portfolios with the best Sharpe Ratio" )
plt.scatter(x = highest_sharpe_ratio.iloc[0,:]['Standard Deviation'], y =
↳ highest_sharpe_ratio.iloc[0,:]['Returns'], color = 'black', alpha = 0.8,
↳ label = "Best Sharpe Ratio portfolio" )
plt.scatter(x = (risk_free / 252).std(), y = risk_free_rate, color = 'black',
↳ alpha = 0.8, marker = '*', label = "Risk Free asset", s = 105)
plt.plot(sigmas, CML, color = 'black', label = "Capital Market Line", alpha = 0.
↳ 7)
```

```
plt.title("Efficient Frontier")
plt.xlabel("Annual Portfolio Standard Deviation")
plt.ylabel("Annual Portfolio Returns")
plt.legend()
plt.show()
```

First of all, we need to observe that **the slope of the CML line is the Sharpe ratio** of the tangency portfolio and that the CML represents a crucial improvement over the efficient frontier. But how are the portfolios of the CML constructed? Let's rearrange the CML formulation as such:

$$r_{\text{CML}} = r_f \left(1 - \frac{\sigma_P}{\sigma_T} \right) + r_T \left(\frac{\sigma_P}{\sigma_T} \right)$$

Simply by combining the risk-free asset with the tangency portfolio and assigning the next weights:

* Tangency portfolio weight: $w_T = \frac{\sigma_P}{\sigma_T}$ * Risk-free weight: $w_f = 1 - \frac{\sigma_P}{\sigma_T}$

There will be more on this in the Portfolio Management course.

1.5 5. Value at Risk (VaR)

Knowing the variance and standard deviation is a big help when it comes to risk. Doing so can help us calculate one of the several ways we can measure risk by using **Value at Risk**.

Value at Risk is one of the easiest risk metrics to interpret. So far, the metrics we have introduced quantify risk as a percentage, in the case of standard deviation, or in units, as the Sharpe ratio does. Value at Risk answers the fundamental question many investors have on their mind: how much can I lose on an investment in the worst-case scenario? This is measured in dollars for the purposes of this class.

Value at Risk measures the potential loss in value of an asset/portfolio over a defined time period. Basically, you will always need to specify the time period and confidence interval that accompanies a Value at Risk. For example, if the VaR of a portfolio is \$1,000,000 over a yearly time period with a 99% confidence interval, it would mean that the portfolio only has a 1% chance of losing more than \$1,000,000 for any given year. VaR has become ubiquitous over the years; every investment bank and risk management firm employs some form of VaR to help keep a cap on the potential losses one can incur. The focus on VaR is very much about downside risk unlike something like standard deviation, which looks at both the upside and downside risk.

There are three basic methods for calculating VaR, each with their own advantages and disadvantages. These build on some of the lessons from earlier, like variance and covariance. Keep in mind that there are countless variations of each basic method, but we will stick with these main three for now:

- Historical Method
- Parametric Method
- Monte Carlo Simulation

1.6 6. VaR Using the Historical Method

This is probably the simplest and most intuitive method of calculating Value at Risk. In short, historical returns are sorted from lowest to highest on an asset or portfolio. Let's say you want to calculate the daily Value at Risk on an equity with a 95% confidence interval. Assuming we can look at the last thousand days of data for this stock, we would take the daily returns and sort them from lowest to highest. From here, we would take the return from the 5th percentile of the data. In this case, with 1,000 days of data, it would be the 50th (0.05×1000) worst daily return from these thousand days. Let's say the 50th worst day had a -4% return. From this, we can assume that the daily VaR for this stock with a 95% confidence interval is -4% . Building on that, if we were to invest $\$1,000$ in said stock, we would expect the worst daily loss to be:

$$-0.04 \times 1000 = -\$40 \text{ with a 95\% confidence interval}$$

Considerations for this method:

- This method uses historical returns to measure VaR empirically, which means that there are no assumptions made about the distribution whereas many models assume the normal distribution.
- Each day for this method is given equal weight, which means if there is a trend in the volatility, you could be overstating or understating the VaR depending on whether the volatility trend is down or up, respectively. One refinement to combat this could be to place greater weight on more recent data.
- Past data does not necessarily indicate what will happen in the future. While the other methods also rely on historical data to a certain extent, this method is solely derived from past historical returns. There are many unforeseen events that can happen, which can change the course of a stock's trajectory and cause the stock to trade differently than it did in the past.

1.6.1 6.1 Implementing Value at Risk (VaR) - Historical Method

Calculating daily historical VaR can be done pretty simply in Python. The order of steps needed is as follows: 1. Calculate all daily returns. 2. Sort these returns from least to greatest. 3. Based on a given confidence level, return the corresponding percentile return. In other words, if we want to calculate daily VaR at a 95% confidence level and we are using 100 data points, the 5th smallest return of that sample would be considered our 95% VaR.

Let's visualize this first with a histogram using Bitcoin.

```
[ ]: # Figure 1
btcusd = yf.download("BTC-USD", start="2014-09-18", end="2022-12-31",
    ↪auto_adjust = False)['Adj Close']
btcusd_returns = btcusd.pct_change().dropna()

plt.hist(x = btcusd_returns, bins=50, log = True, alpha = 0.7)
plt.xlabel('Daily Returns')
plt.ylabel('Frequency')
plt.title('Histogram of BTCUSD Daily Returns')
plt.show()
```

In order to have a better visualization of the extreme events, we used a logarithmic scale for the y axis. Let's calculate the historical VaR from this data.

The next function `getHistoricalVar()` takes two arguments: 1. a series of daily returns 2. a confidence level, 95 for example, would correspond to 95% confidence

```
[ ]: def getHistoricalVar(returns, confidenceLevel):
    var = np.quantile(returns, 1 - confidenceLevel)
    print(
        f"The Historical VaR with confidence {confidenceLevel} is {round(100 *
↪var, 3)}% \n"
    )

getHistoricalVar(btcusd_returns, 0.90)
getHistoricalVar(btcusd_returns, 0.95)
getHistoricalVar(btcusd_returns, 0.99)
```

After running the function, we can see that the greater we make our confidence level, the lower the Value at Risk will be. This is a useful tool when determining how much a financial asset can lose over a certain time period.

You may be wondering now, since we only use a 95% confidence level, what happens on those 5% of days where losses exceed -6.02%. We can use another handy metric, Conditional Value at Risk (CVaR), to deal with these situations.

1.6.2 6.2 Conditional Value at Risk (CVaR) using Historical Data

Conditional Value at Risk (CVaR), also known as Expected Shortfall (ES), is a widely used risk measure in financial risk management. While Value at Risk (VaR) estimates the loss over a specific time horizon given a confidence level (e.g., 99%), CVaR goes beyond VaR by providing the expected loss if the VaR is exceeded. In other words, CVaR calculates the average loss in the tail of the distribution beyond the VaR threshold.

Mathematically, the CVaR at confidence level α is:

$$\text{CVaR}_\alpha = \mathbb{E}[L | L > \text{VaR}_\alpha]$$

where L represents the losses.

Historical CVaR is a non-parametric approach, which means that it does not rely on distribution assumptions for the returns. Instead, we can directly calculate the expectation of the losses after we have obtained the VaR. In practice, we can calculate the mean of the returns that are located to the left of the VaR.

```
[ ]: def getHistoricalCVar(returns, confidenceLevel):
    var = np.quantile(returns, 1 - confidenceLevel)
    cvar = returns[returns <= var].mean()
    print(
        f"With {confidenceLevel} percent confidence VaR, our Expected Shortfall
↪is {round(100 * cvar, 3)} using historical VaR \n"
```

```
)

getHistoricalCVar(btcusd_returns, 0.90)
getHistoricalCVar(btcusd_returns, 0.95)
getHistoricalCVar(btcusd_returns, 0.99)
```

1.7 7. VaR Using the Parametric Method

Parametric VaR is a method of calculating Value at Risk assuming that the returns follow a specific probability distribution. In the standard variation, a normal distribution is assumed, but one can use other distributions as well, as we will see in the second example where we assume a t-distribution.

1.7.1 7.1 Normal Distribution Parametric VaR

In the case of a normal distribution, the use of a Z-table is essential in calculating the VaR. While performing the task, we need to always take into account that the risk refers to the **left tail**.

$$\text{VaR} = \mu + \sigma Z_{\alpha}$$

where * μ is the mean return * σ is the standard deviation of returns * Z_{α} is the Z-score that corresponds to the confidence interval

Considerations for this method:

- If returns are not normally distributed, you will likely be underestimating the true Value at Risk. For example, many stocks have more outliers than the normal distribution would assume; this means the computed Value at Risk will be lower than what it is in actuality.
- Variance and covariance between return streams must be considered when computing VaR for a portfolio. Even if returns are normally distributed, the VaR calculation can still be thrown off if the estimated variances and covariances are incorrect. This can be further amplified if variances and covariances are changing over time.
- Models that allow variance to change over time (heteroskedasticity) display a greater degree of accuracy. Engle has argued that autoregressive conditional heteroskedasticity (ARCH) and generalized autoregressive conditional heteroskedasticity (GARCH) models provide better forecasts of variance and, by extension, better measures of Value at Risk.
- This method breaks down whenever a portfolio has assets with a non-linear payoff structure, e.g., options.

1.7.2 7.2 Implementing Parametric VaR with Normal Distribution

While it's well-established that financial returns, particularly for equities, often exhibit non-normal characteristics, such as skewness and fat tails, this parametric approach still holds significant value in financial risk management. The normal distribution assumption provides an efficient way to estimate risk, making it highly useful in situations where quick approximations are needed or can be used in conjunction with other models.

To calculate Value at Risk (VaR) using the parametric method, we will leverage the `norm.ppf()` function from Python's `scipy.stats` module. The percent point function (ppf) is actually the

inverse cumulative distribution function (CDF), and it allows us to compute the quantile that corresponds to a given confidence level.

```
[ ]: mean = btcusd_returns.mean()
std = btcusd_returns.std()

var_90 = stats.norm.ppf(0.1, mean, std)[0]
var_95 = stats.norm.ppf(0.05, mean, std)[0]
var_99 = stats.norm.ppf(0.01, mean, std)[0]

print(f"The parametric VaR assuming a normal distribution with a 90% confidence_↵
↵interval, is {round(100 * var_90, 3)} \n")
print(f"The parametric VaR assuming a normal distribution with a 95% confidence_↵
↵interval, is {round(100 * var_95, 3)} \n")
print(f"The parametric VaR assuming a normal distribution with a 99% confidence_↵
↵interval, is {round(100 * var_99, 3)} \n")
```

Excess kurtosis observed in asset return distributions indicates that the normal distribution is not the best choice to represent the returns, especially in the tails. Their fat-tailed nature indicates a higher probability of extreme outcomes, in many cases a lot higher than what a normal distribution predicts. The latter forces us to employ a heavy-tailed distribution like the Student's t-distribution.

Exercise 6

Calculate the Kurtosis of BTCUSD returns and make sure you understand the effects of high kurtosis on the tails. Then argue about the reason the normal distribution seems to underestimate the VaRs in comparison to the historical method.

Exercise 7 (discussion)

Check the
BTCUSD
histogram
presented in
Figure 1. Make
sure you also
print the first 10
returns after you
have sorted them
with `ascending`
`= True`.
Elaborate on the
gaps between the
returns (as data
points) of the left
tail: does our
sample
adequately
represent the left
tail?

In the next section, we will delve into how the Student's t-distribution can be utilized in Value at Risk (VaR) calculations, offering a more accurate assessment of risk for assets exhibiting fat tails.

1.7.3 7.3 Implementing Parametric VaR with *t*-distribution

The Student's t-distribution is commonly used as an alternative to the normal distribution when modeling data with fat tails. The degrees of freedom parameter controls the heaviness of the tails. As the degrees of freedom increase, the t-distribution approaches the normal distribution. For smaller degrees of freedom (dof), the tails become heavier, indicating a higher probability of extreme values.

```
[ ]: # degrees of freedom

def getTVar(returns, dof, confidenceLevel):
    mean = returns.mean()
    std = returns.std()
    var = np.sqrt((dof - 2) / dof) * stats.t.ppf(1 - confidenceLevel, dof) *
    ↪std + mean
    return (100 * var).round(3)

print(f"The parametric VaR using t-distribution with a confidence interval 0.9, ↪
    ↪is {getTVar(btcusd_returns, 5, 0.9)}% \n")
print(f"The parametric VaR using t-distribution with a confidence interval 0.
    ↪95, is {getTVar(btcusd_returns, 5, 0.95)}% \n")
```

```
print(f"The parametric VaR using t-distribution with a confidence interval 0.
↪99, is {getTVar(btcusd_returns, 5, 0.99)}% \n")
```

1.8 8. VaR Using Monte Carlo Simulation

In the previous sections, we discussed how one can use historical data on financial series to compute the Value at Risk. But what if we have incomplete or insufficient data to do so? In this case, we can hypothesize a distribution that generates the data and run simulations using those distributional assumptions. This method is called *Monte Carlo VaR*.

The Monte Carlo VaR includes a series of simulations, where each return stream is represented as a random variable. This variable can be taken from any probability distribution, which is great because that means it doesn't necessarily assume a normal distribution. There is a lot of flexibility in choosing what kind of distribution to use. All the variables are then dollar-weighted and simulated to see what the total portfolio value is at the end of each run. These simulation returns are then sorted lowest to highest, and we can easily look to see what the Value at Risk is using similar computations to the historical method, except this time, we're using simulated returns instead of historical returns. For example, if you ran a series of 1,000 simulations, you would look at the 50th lowest value to determine the VaR for a 95% confidence interval.

Considerations for this method: * Estimations will not be effective if the probability distributions used to determine the random variables are incorrect. Many use past data to get an idea of what the probability distribution should be; this method allows for some subjectivity. * You can estimate VaR more effectively for portfolios containing options with this method versus the parametric method since Monte Carlo doesn't assume a normal distribution of returns.

```
[ ]: # Parameters
initial_investment = 100000 # Initial portfolio value (in dollars)
mean_return = 0.0005 # Daily mean return of the asset (e.g., 0.05%)
volatility = 0.02 # Daily volatility (not annual!) is 2%
days = 1 # 1 day time horizon
num_simulations = 10000 # Number of Monte Carlo simulations
confidence_level = 0.95 # Confidence level for VaR (e.g., 95%)

# Generate random returns based on normal distribution
np.random.seed(2024) # For reproducibility
random_returns = np.random.normal(loc=mean_return * days, scale=volatility * np.
↪sqrt(days), size=num_simulations)

# Calculate portfolio values at the end of the simulation
portfolio_end_values = initial_investment * (1 + random_returns)

# Calculate profit and loss (P&L)
portfolio_pnl = portfolio_end_values - initial_investment

# Calculate the VaR at the specified confidence level
VaR = np.percentile(portfolio_pnl, (1 - confidence_level) * 100)
```

```

# Output the results
print(f"Value at Risk (VaR) at {confidence_level * 100}% confidence level:␣
↪-${abs(VaR):,.2f}")

#Plot the distribution of portfolio P&L
plt.figure(figsize=(10, 6))
plt.hist(portfolio_pnl, bins=50, edgecolor='black', alpha=0.7)
plt.axvline(x=VaR, color='r', linestyle='dashed', linewidth=2)
plt.title(f"Monte Carlo Simulation of Portfolio P&L (VaR at {confidence_level *␣
↪100}% Confidence Level)")
plt.xlabel('Profit & Loss ($)')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()

```

1.9 9. Conclusion

In this lesson, we've introduced a wide range of important concepts in financial risk analysis. We started by examining different types of returns, including total returns, dividend-adjusted returns, and excess returns, which form the foundation for more advanced risk metrics. We then explored the Sharpe Ratio and showcased how important this ratio is in the portfolio management context.

The latter part of the lesson focused on downside risk metrics, particularly Value at Risk (VaR). We covered various methods for calculating VaR, including historical, parametric (with both normal and t-distributions), and Monte Carlo simulation approaches. We also introduced the concept of Conditional Value at Risk (CVaR) as an extension of VaR.

References

- Elton, Edwin J., et al. *Modern Portfolio Theory and Investment Analysis*. John Wiley & Sons, 2009.
- Engle, Robert. "GARCH 101: The Use of ARCH/GARCH Models in Applied Econometrics." *The Journal of Economic Perspectives*, vol. 15, no. 4, 2001, pp. 157–168.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Barndorff-Nielsen, Ole E., et al. "Realized Kernels in Practice: Trades and Quotes." *Econometrics Journal*, vol. 12, 2009, pp. C1-C32. <https://onlinelibrary.wiley.com/doi/epdf/10.1111/j.1368-423X.2008.00275.x>.

- **Required pages:** C7-C8
- **Estimated reading time:** 30 minutes

2. Brownlees, Christian T., and Giampiero M. Gallo. "Financial Econometric Analysis at Ultra-High Frequency: Data Handling Concerns." *Computational Statistics & Data Analysis*, vol. 51, no. 4, 2006, pp. 2232-2245. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=886204.

- **Required pages:** Sections 3.1-3.2
- **Estimated reading time:** 15 minutes

Tick Data

Reading Time	6h
Prior Knowledge	financial time series, pandas, matplotlib
Keywords	tick data, Level 1 Data, TOB, Kurtosis, Order Flow Imbalance, resample, trade, quote

In this lesson, we will work directly with tick data, the most granular form of financial market information. We'll start with the basics: what tick data is, why it's called that, and why it's so important. Then, we'll get our hands dirty with real datasets, learning how to clean and process this high-resolution market information. Why does this matter? Well, as financial markets become increasingly automated, understanding high-frequency data is becoming crucial for anyone serious about quantitative finance or algorithmic trading. We'll be working with actual tick data from cryptocurrency exchanges and stock markets. It is messy and challenging, but that's part of this workflow. By the end of this lesson, you'll have practical experience in handling this data, from cleaning it up to creating the time-aggregated charts you're probably more familiar with.

1. What is Tick Data?

In financial markets, data is the means that drives decision-making and trading strategies. Among the various types of market data available, tick data is the most granular and comprehensive. Tick data represents the most detailed record of trading activity, including every single transaction that occurs on an exchange. It includes the price, volume, and timestamp of each trade (and depending on the exchange or data vendor, there could be additional columns), allowing market participants to analyze market dynamics at the most fundamental level.

But why do we use the term "tick"?

The price of an asset seems to be a continuous variable. Indeed, in watching a plot of an asset someone can infer that the price can take any number between price point A and B. But this is not the case. The price is a discrete variable that moves in increments that are a product of a small increment called "tick".

Essentially, the word "tick" refers to the smallest price movement of an asset (stock, commodity, or futures) and it can be different for the same asset across exchanges and brokers (Harris). For example, if the tick size of a stock is $\$0.01$ and the price of the stock is $\$100$, the price can only be $\$100 + \lambda * 0.01$ with $\lambda \in \mathbb{Z}$. We can have prices of $\$100.01$ or $\$100.15$ but never $\$100.005$.

Historically, the term "tick" originates from the sound of the early stock ticker machines of the late 19th century, which printed stock prices on a ticker tape.

Crash of 1929 Ticker Tape

NEW YORK STOCK EXCHANGE OCT 29 1929									
ST	IL	PVX	TOH	SDS	KYRPR	SS	UFG	WNO	RDE
20.65	25.129	2.104	2200.20	25.58	8.105	20.78	50.106	10.30	1800.11
YV	X	WCO	RDPD	STU	AFW	CSU	PSTPR	WA	NE
8.70	15.55	8.10	4.50	30.00	30.40	49.10	10.07	69.15	10.1
R	LAST. LO	PFR	BMT	BK	DEL	GT	GIS	RY	NA
50.52	10.30	5.1	50.52	15.00	55.22	56.9	17.69	46.80	20.0
PU	CHC	KVR	SUR	XPR	SSH	JMP	HX	YC	RJB
3.60	1.78	30.35	14.58	50.1	35.14	50.1	60.128	17.48	10.00
H	CV.N	IN	K	H.KRT	LO	X	NSOCT	NCW	PRC
5.100	2.28	10.28	70.20	50.01	35.20	20.00	28.2	2.58	340.1
SC	ADT	AFW	OSH	RGO	X	STU	X	KRT	MRC
20.00	26.25	25.00	30.370	3.20	3.20	190.58	3.50	2.58	10.58
NPL	R	RAY	SGU	WX	CV.N	H	S	K	RBC
30.00	25.3	3.39	15.34	50.24	25.00	130.1	24.28	85.4	553.34
AC	ARK	TS	TFS	TCO	K	R	J	WM	N
3	10.00	130.20	33.4	30.00	52.20	1.12	50.12	10.5	455.4

As stock ticker machines recorded the stock prices, each transaction was accompanied by a ticking sound.

Adapted from: Museum of American Finance. "Crash of 1929 Ticker Tape."

<https://www.moaf.org/publications-collections/museum-collection/objects/crash-1929-ticker-tape>

"Tick data" is known by many names, each unveiling its important characteristics:

- **Tick-by-tick data:** Each point is a tick.
- **(Ultra) High-Frequency data:** The frequency of the ticks and the volume traded are very high, especially in liquid markets.
- **Trades:** Each tick represents a trade.
- **Transactions:** Each trade is a transaction between two parties.

- **Real-time data:** This data can be delivered in real-time with minimal latency.

Tick data not only includes trade data but may also include quote data. This data is known as a limit order book. So, if a market maker raised the AMOUNT they were willing to buy and placed an order at the bid side, there would be an additional observation that changes the bid size.

Likewise, if a market maker raised the bid price (the price where they were willing to buy), that would be an additional observation. Even if there were no trades, limit order book data is a rich source of information that contains the intentions of participants. As you will see in Figure 7, the intentions (new bids, asks, cancellations, etc.) change faster than the actions (actual trades) of the participants.

By now, you might have already some exposure to financial data. Most people have seen candlestick charts and price charts with a fixed time interval on the x -axis. There are online applications such as TradingView or Yahoo Finance and many online exchanges where one can plot the stock of their choice at any fixed time interval and even use their favorite indicators. Essentially, what you see is a structured dataset created after some people/code behind the scenes have acquired the tick data and produced the time bars.

The purpose of this lesson is not only to introduce tick data to students, showing how the data looks when it is first produced, but also to guide you in creating and plotting the structured datasets that you have been using until now. By doing so, we will transition from production (tick data) to consumption-ready datasets, similar to those that will be used in later courses for modeling and analysis.

It should be clear by now that tick data (and even more so the high-frequency order book) consists of the most granular data you can find. Being somewhat naive, one could say that this is all the data that is produced as trading goes on, and therefore, we can expect that it contains all the possible information. On the other hand, someone could argue that there are many external factors such as macroeconomic trends, sentiment, regulations, news, and more. However, if you are a believer in the efficient market hypothesis, then you should expect that any external factor will be priced in immediately. Thus, the external information becomes part of the price and the trading data in general, as soon as it becomes known.

Since tick data can be delivered in real-time, traders who analyze this data instead of minute, hourly, or daily time bars (which will incorporate the information in one minute, one hour, or a day, respectively) should be able to extract the information before the rest of the participants. These traders, though, face a very difficult task: separating information from noise.

2. Examples of Tick Data

For the purposes of this lesson, we chose three different high-frequency datasets. Two of them consist of tick-by-tick trade data, and the third one is Level 1 data mixed with trades. The choice of the assets is random and mostly affected by the availability of such data. Almost always, the high-frequency data is not given for free and is rather expensive. This is normal considering the vast amount of data produced daily and the technical difficulties of storing and distributing it.

For this reason, we will be exploring:

1.) BTC/USDT tick-by-tick trades downloaded from Kraken exchange:

<https://support.kraken.com/hc/en-us/articles/360047543791-Downloadable-historical-market-data-time-and-sales->

2.) RVN/USDT tick-by-tick trades downloaded from Binance exchange:

<https://data.binance.vision/>

3.) AGLJ/ZAR Level 1 Data with trades downloaded from:

<https://data.mendeley.com/datasets/4rrk89c3b2/2>

Due to VM restrictions, a very small portion of the data above is sampled for teaching purposes. But you are encouraged to download large tick-by-tick datasets locally in order to face firsthand the technical difficulties of dealing with big data.

Tick-by-tick trade data consists of the following columns:

- Symbol: the unique identifier of the financial instrument.
- Timestamp: the exact time when the transaction was recorded (granularity ranges from second to nanosecond).
- Price: the executed price of the trade.
- Volume: The number of units traded.
- Trade Direction (optional): indicates whether the trade was buyer-initiated or seller-initiated.

The most important aspect of tick data, which is granularity, allows us to undertake:

- Studying intraday volatility.
- Understanding price formation more thoroughly.
- Backtesting of high-frequency (HF) trading strategies.
- Assessing market impact to determine scaling a strategy.
- Analyzing the transaction costs of a strategy.

- Exploring execution strategies.
- Detecting market manipulation (can be used by regulatory authorities).
- Aggregation to time-based or volume-based bars.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

```
In [2]: # Load the data, dataset (1) - BTC/USD from Kraken
btc_usdt = pd.read_csv("XBTUSD.csv", names=['UNIX_time', 'price', 'volume'])
btc_usdt.head()
```

```
Out[2]:
```

	UNIX_time	price	volume
0	1688169604	30474.1	0.018800
1	1688169604	30474.8	0.000103
2	1688169604	30476.5	0.014997
3	1688169656	30483.5	0.000614
4	1688170008	30479.0	0.000328

The dataset above is an example of a tick-by-tick trades dataset, and it consists of all the trades that occurred in a specified interval. The first thing that you notice is the UNIX_time column, which does resemble time or date! This timestamp is called "UNIX time," and it counts the number of time units (seconds, milliseconds, microseconds, and nanoseconds) from 00:00:00 UTC January 1, 1970, until now.

This time representation provides:

- Standardization: universal representation of time.
- Simplicity: it is a single integer.
- Efficiency: calculations are straightforward and computationally efficient.

The Unix Timestamps can be represented with different levels of precision:

- 10 digits: Represents seconds (e.g., 1688169604).
- 13 digits: Includes milliseconds (e.g., 1688169604123).
- 16 digits: Includes microseconds (e.g., 1688169604123456).
- 19 digits: Includes nanoseconds (e.g., 1688169604123456789).

You can also check [this](#) website.

In the dataset (1), we can count 10 digits for the UNIX_time column. Therefore, the precision is given in seconds, and this is the only thing we need to know in order to transform Unix time into human-readable datetime.

```
In [3]: # Convert the UNIX timestamp to human readable datetime. Check the argument unit='s'
btc_usdt['timestamp'] = pd.to_datetime(btc_usdt['UNIX_time'], unit='s')
btc_usdt.set_index('timestamp', inplace=True) # Set the timestamp as the index
btc_usdt.head()
```

```
Out[3]:
```

	UNIX_time	price	volume
	timestamp		
	2023-07-01 00:00:04	1688169604	30474.1 0.018800
	2023-07-01 00:00:04	1688169604	30474.8 0.000103
	2023-07-01 00:00:04	1688169604	30476.5 0.014997
	2023-07-01 00:00:56	1688169656	30483.5 0.000614
	2023-07-01 00:06:48	1688170008	30479.0 0.000328

3. Visualizing Tick Trade Data

Now, let's examine our dataset visually. Visualizing tick data presents unique challenges due to its extremely high resolution. Consider this analogy: imagine you have an ultra-high-resolution image displayed on a small screen or viewed from a distance. Much of the detail is lost, and you might wonder about the value of such high resolution. Similarly, when plotting tick data, we risk obscuring important patterns if we simply plot every point. The key is to find ways to represent this high-frequency data that reveal its inherent patterns and characteristics without overwhelming the viewer. In this section, our data is sourced from Gebbie and Nonyane's "TRTH JSE AGLJJ Intraday Transaction Test Data" and from the datasets mentioned in section 2.

```
In [4]: #Figure 1
fig, ax = plt.subplots(figsize=(12, 5))
ax.scatter(btc_usdt.index, btc_usdt['price'], s = 0.001, alpha=0.4)
ax.set_ylabel('Price')
ax.set_title('BTC/USDT Price')
ax.set_ylim(20000, 32000)

custom_legend = plt.Line2D([0], [0], linestyle="none", marker='o', color='b', marke
ax.legend(handles=[custom_legend], loc='upper right')

ax2 = ax.twinx() # Create a second y-axis
ax2.plot(btc_usdt.index, btc_usdt['volume'].rolling(10000).sum(), color='red', labe
ax2.set_ylabel('Volume: Rolling sum')
```

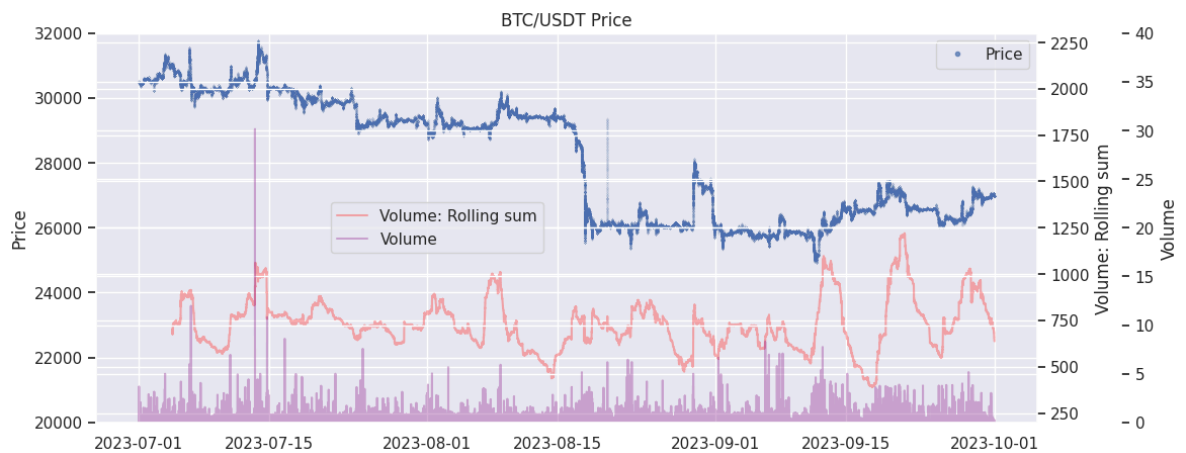
```

ax2.set_ylim(200, 2300)

ax3 = ax.twinx() # Create a third y-axis
ax3.plot(btc_usdt.index, btc_usdt['volume'], color='purple', label='Volume', alpha=
ax3.set_ylabel('Volume')
ax3.set_ylim(0, 40)
ax3.spines['right'].set_position(('outward', 60))

fig.legend(bbox_to_anchor=(0.5,0.56)) # Adjust the legend position
plt.show()

```



We must note that datasets (1) and (2) are retrieved directly from the exchanges where they were produced; thus, we expect no missing inputs. That does not mean there are no errors in the data!

Let's explore our dataset:

```

In [5]: between_trades_interval = btc_usdt['UNIX_time'].diff().dropna() # Calculate the tra
print("Summary Statistics \n\n", between_trades_interval.describe(), "\n\n") # Get
print("10 Largest trade durations \n\n", between_trades_interval.sort_values(ascend

```

Summary Statistics

```
count      358309.000000
mean        22.184026
std         55.319270
min          0.000000
25%          0.000000
50%          0.000000
75%         20.000000
max         2195.000000
Name: UNIX_time, dtype: float64
```

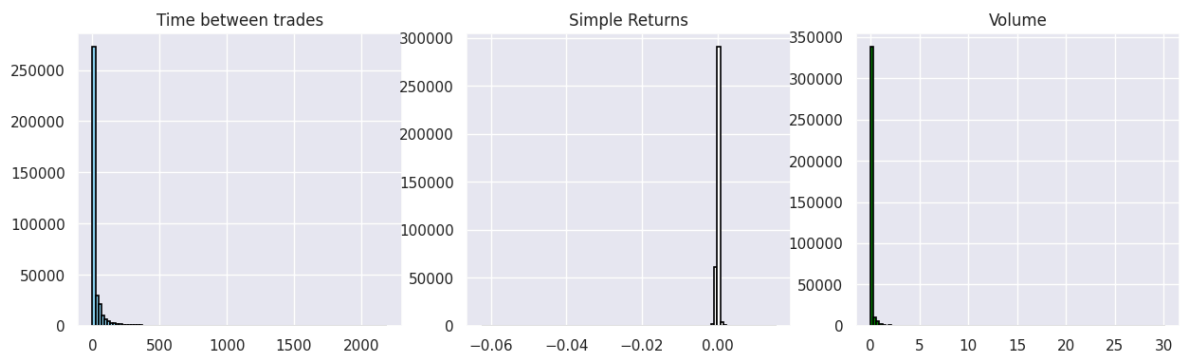
10 Largest trade durations

```
timestamp
2023-08-01 15:41:45    2195.0
2023-07-12 09:19:13    2001.0
2023-08-17 10:30:32    1809.0
2023-08-18 15:05:14    1576.0
2023-07-23 07:29:23    1281.0
2023-07-23 02:13:51    1244.0
2023-08-06 03:40:46    1224.0
2023-09-28 15:28:56    1177.0
2023-07-20 23:44:23    1125.0
2023-07-16 03:52:08    1113.0
Name: UNIX_time, dtype: float64
```

Judging from the above results (high trade durations), the pair BTC/USDT in the specific sample seems to be illiquid. But we know that liquidity can vary greatly over time and this sample may not be representative of this pair's liquidity on this exchange. To draw more robust conclusions, we would need to analyze a larger dataset spanning different time periods and market conditions.

Let's check the distribution of volume, simple returns and intervals between trades.

```
In [6]: #Figure 2
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4))
ax1.hist(between_trades_interval, bins=100, color='skyblue', edgecolor='black', lin
ax1.set_title("Time between trades")
ax2.hist(btc_usdt['price'].pct_change().dropna(), bins=100, color='white', edgecolo
ax2.set_title("Simple Returns")
ax3.hist(btc_usdt['volume'], bins=100, color='green', edgecolor='black', linewidth=
ax3.set_title("Volume")
plt.show()
```



The above results hint at three important facts:

- There are several trades that happened at the same second. Those trades could be a single trade that was divided in many trades at the moment of execution as the volume at the best bid or ask was not enough to cover the trade volume in that instant. These trades are called **split transactions**.
- There are several trades that did not result in a price change.
- A lot of trades seem to have very low volume.

```
In [7]: boolean_array_is_duplicate = btc_usdt['UNIX_time'].duplicated(keep=False) # Check for
sum_of_duplicates = boolean_array_is_duplicate.sum() # Sum the duplicates
print("The number of trades that occurred at the same second with at least one more
print("The percentage of these trades to the number of all trades, is: ", round(sum_
print("The percentage of the volume of these trades to the entire volume, is: ", rou
```

The number of trades that occurred at the same second with at least one more trade, is: 241814

The percentage of these trades to the number of all trades, is: 67.49 %

The percentage of the volume of these trades to the entire volume, is: 90.44 %

We will assume that these trades are split transactions. After some simple aggregations, the histograms plotted above look very different:

```
In [8]: # Figure 3
aggr_dataset_1 = pd.DataFrame()
aggr_dataset_1['UNIX_time'] = btc_usdt['UNIX_time'].groupby(btc_usdt['UNIX_time']).
aggr_dataset_1['volume'] = btc_usdt['volume'].groupby(btc_usdt['UNIX_time']).sum()
aggr_dataset_1['price'] = (btc_usdt['price'] * btc_usdt['volume']).groupby(btc_usdt
aggr_dataset_1.index = btc_usdt.index.drop_duplicates() # Drop the duplicates from

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4))
ax1.hist(aggr_dataset_1['UNIX_time'].diff().dropna(), bins=100, color='skyblue', ed
ax1.set_title("Time between trades")
ax2.hist(aggr_dataset_1['price'].pct_change().dropna(), bins=100, color='white', ed
ax2.set_title("Simple Returns")
ax3.hist(aggr_dataset_1['volume'], bins=100, color='green', edgecolor='black', line
ax3.set_title("Volume")
plt.show()
```



At this point, one could continue the workflow with a returns analysis, an order flow imbalance (OFI), volatility estimation, liquidity measure, etc. Since these datasets will be used for tick-data presentation and not analysis, we will move to dataset (2), and an introductory workflow will be presented with dataset (3).

4. Including the Trade Direction

Let's check dataset (2), which holds the RVN/USDT trades but with one addition to the dataset (1): there is an additional column, "Trade Direction," which holds information on whether the trader who initiated the trade was a buyer or a seller. In our case, the exchange is storing a boolean value (True, False) for each row that answers the question *is the buyer the market maker?*

As we know, for each transaction, there must be a buyer and a seller. If the answer to the above question is true, then the market maker placed a bid order in the orderbook with the intention to buy, and a seller initiated the trade (partially?) filling that buying order. If, on the other hand, the answer is false, then the market maker was a seller who placed an ask order. In this case, the initiator of the trade is a buyer who filled that specific ask order.

This additional boolean array is a source of valuable information.

We will begin by loading dataset (2). In many cases, the data vendor/exchange will let you download the data in `csv` files with each file containing the data of one day (or week, or month, or custom interval). In our case, dataset (2) consists of four daily files. We will be using the `glob` library due to its simplicity in selecting the files we need:

<https://builtin.com/software-engineering-perspectives/glob-in-python>.

```
In [9]: import glob

# Load the data, dataset (2) - RVN/USDT
list_of_files = glob.glob('RVNUSDT*') # List the RVNUSDT files in the directory
column_names = ['id', 'price', 'qty', 'base_qty', 'UNIX_time', 'is_buyer_maker', 'i
rvn_usdt = pd.concat([pd.read_csv(file, names=column_names) for file in list_of_fil
rvn_usdt.sort_values('UNIX_time', inplace=True) # Sort the values by time
rvn_usdt.head(5)
```

Out[9]:

	id	price	qty	base_qty	UNIX_time	is_buyer_maker	is_best_m
1322916	54407520	0.02588	424.9	10.996412	1680307215745	False	
1322917	54407521	0.02588	3313.5	85.753380	1680307215759	False	
1322918	54407522	0.02588	4172.5	107.984300	1680307215759	False	
1322919	54407523	0.02588	5162.5	133.605500	1680307215762	False	
1322921	54407525	0.02589	1105.3	28.616217	1680307215762	False	

Let's step back for a moment. Dataset (2) is a tick-by-tick trades dataset, but there are twice as many columns as in dataset (1). At this point, it must be clear that depending on the data vendor and/or the exchange, there could be additional columns to the three basic ones (price, volume, and timestamp). In our case, the only meaningful addition to the basic columns is the `is_buyer_maker` column.

The `base_qty` represents the volume of the base currency (the currency that the asset is traded against) traded given the volume of the asset traded and the price of the trade $\text{base_qty} = \text{qty} * \text{price}$. The column `id` represents the id of each trade and can be used to filter out duplicate entries. Lastly, the column `is_best_match` is a boolean array that indicates whether the trade was executed at the best possible price given the orderbook at that time (True or False).

We will check whether the additional columns hold any information before we omit them in order to handle a smaller dataset, which will be less taxing on ram.

```
In [10]: print("Number of duplicate entries: ", rvn_usdt['id'].duplicated().sum()) # Check for
print("Number of entries that were not executed at the best possible price: ", (~rvn_usdt['is_best_match']).sum())
print("Can the base_qty be derived from the qty and price?: ", np.isclose(rvn_usdt['base_qty'], rvn_usdt['qty'] * rvn_usdt['price']).all())
print("Due to the results above, we can drop the columns 'id', 'is_best_match' and 'base_qty'")

rvn_usdt = rvn_usdt.drop(columns=['id', 'is_best_match', 'base_qty']) # Drop the un
rvn_usdt['timestamp'] = pd.to_datetime(rvn_usdt['UNIX_time'], unit='ms') # Convert
rvn_usdt.set_index('timestamp', inplace=True) # Set the timestamp as the index
rvn_usdt.head()
```

```
Number of duplicate entries: 0
Number of entries that were not executed at the best possible price: 0
Can the base_qty be derived from the qty and price?: True
Due to the results above, we can drop the columns 'id', 'is_best_match' and 'base_qty'
```

Out[10]:

	price	qty	UNIX_time	is_buyer_maker
timestamp				
2023-04-01 00:00:15.745	0.02588	424.9	1680307215745	False
2023-04-01 00:00:15.759	0.02588	3313.5	1680307215759	False
2023-04-01 00:00:15.759	0.02588	4172.5	1680307215759	False
2023-04-01 00:00:15.762	0.02588	5162.5	1680307215762	False
2023-04-01 00:00:15.762	0.02589	1105.3	1680307215762	False

5. Manipulating Tick Data

From now and on, with "buying volume" or "selling volume," we will refer to the volume of the trades whose initiator/taker was the buyer or seller, respectively. To understand how to use the added information, we will present the data manipulations that are essential for calculating the buying and selling volume.

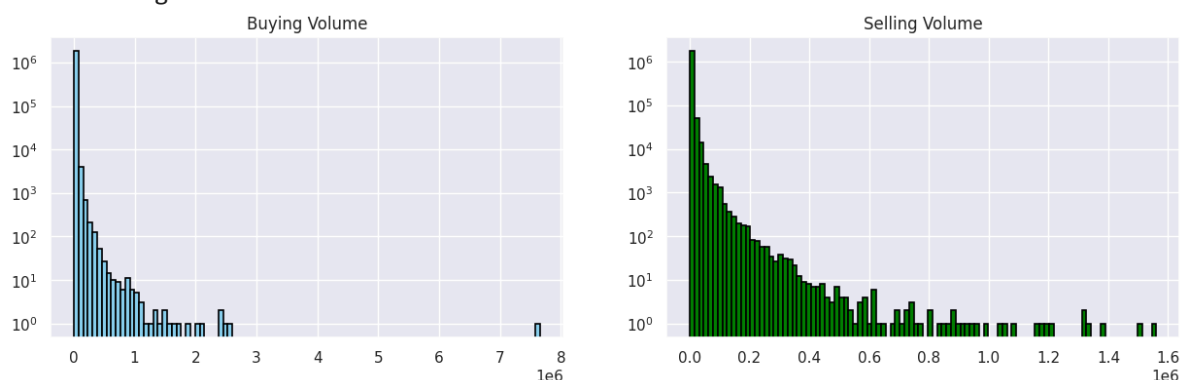
```
In [11]: # Figure 4
buying_volume = rvn_usdt['qty'] * (~rvn_usdt['is_buyer_maker']) # An array that con
selling_volume = rvn_usdt['qty'] * rvn_usdt['is_buyer_maker'] # An array that conta

print("The buying volume is: ", buying_volume.sum()) # Calculate the buying volume
print("The selling volume is: ", selling_volume.sum()) # Calculate the selling volu

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 4))
ax1.hist(buying_volume, log = True, bins=100, color='skyblue', edgecolor='black', l
ax1.set_title("Buying Volume")
ax2.hist(selling_volume, log = True, bins=100, color='green', edgecolor='black', li
ax2.set_title("Selling Volume")
plt.show()
```

The buying volume is: 6081743642.6

The selling volume is: 6530522239.800001



Exercise 1

In the histogram above, we have used the argument `log = True` in order to transform the `price` axis in log scale. To see why, plot the histograms of the buying and selling volume with `log = False`. Furthermore, let's distinguish between transforming the `price` axis for visualization purposes and transforming the data itself, using the natural logarithm. Create the two histograms again with `log = False`, but use the `np.log` function to transform the buying and selling volume: `np.log(buying_volume[buying_volume > 1])`. Do the same for the histograms presented in Figure 3. Explain the differences.

Exercise 2

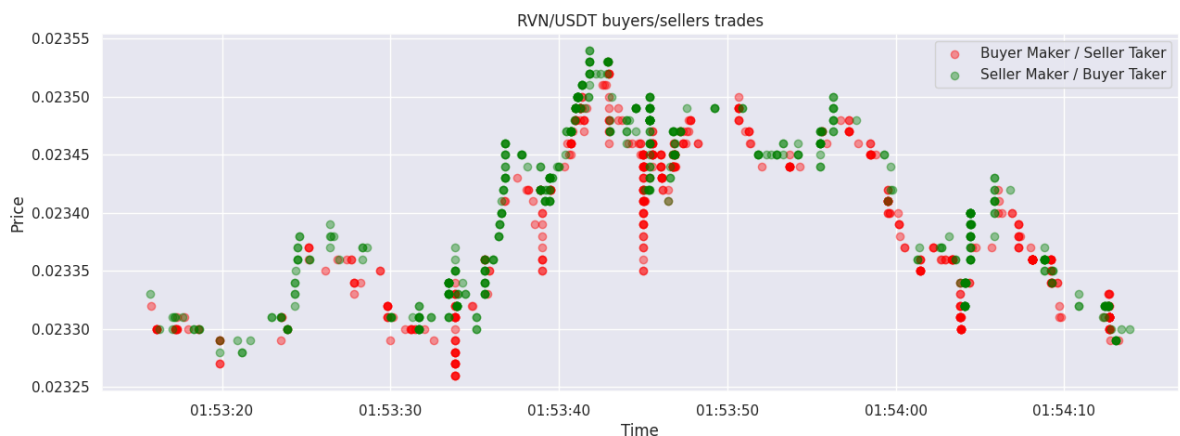
Calculate the tick size for datasets (1) and (2). Explain your methodology in the forum.

We will now plot the buying and selling volume in a visually pleasing way. When plotting high resolution data, it makes no sense to zoom out in order to see the "big picture." On the contrary, we opt for zoomed-in graphs or interactive plots (plotly, bokeh) where we can zoom in on demand.

```
In [12]: # Figure 5
rvn_usdt_slice = rvn_usdt.loc['2023-07-01 01:53:15.745':'2023-07-01 01:54:15.745']

fig, ax = plt.subplots(figsize=(15, 5))

ax.scatter(x = rvn_usdt_slice[rvn_usdt_slice['is_buyer_maker'] == True].index, \
           y = rvn_usdt_slice[rvn_usdt_slice['is_buyer_maker'] == True]['price'], \
           label='Buyer Maker / Seller Taker', alpha = 0.4) # Plot the buyer maker
ax.scatter(x = rvn_usdt_slice[rvn_usdt_slice['is_buyer_maker'] == False].index, \
           y = rvn_usdt_slice[rvn_usdt_slice['is_buyer_maker'] == False]['price'], \
           label='Seller Maker / Buyer Taker', alpha = 0.4) # Plot the seller maker
ax.set_title("RVN/USDT buyers/sellers trades")
ax.set_xlabel("Time")
ax.set_ylabel("Price")
ax.legend()
plt.show()
```



Exercise 3

In Figure 5, we can see a zoomed-in graph of the RVN/USDT buyers' and sellers' trades. The interval chosen is exactly 1 minute. In this exercise, you must recreate Figure 5 for different intervals. You can choose larger intervals and/or different points in time. In order to have a guide in choosing the intervals that **interest** you, you will have to plot the entire price plot of the RVN/USDT pair.

Exercise 4 (Optional)

Using the code provided in this notebook as a guide, download the tick data locally in order to continue your workflow on your own PC. Then, try to use plotly or bokeh (holoviews) in order to create the interactive plots that will let you zoom in without slicing the dataset every time you want to create a new graph. Check [here](#) for the gallery provided by plotly, and make sure you understand the difference between an interactive plot and a static one.

6. Level 1 Data (L1)

Level 1 data (or else top-of-book data (TOB)) typically include the following columns:

- Symbol: the unique identifier of the financial instrument.
- Timestamp: the exact time when the event was recorded.
- Bid price: the highest price a buyer is willing to pay.
- Ask price: the lowest price a seller is willing to accept.
- Ask volume: number of units available at the ask price.
- Bid Volume: number of units available at the bid price.

```
In [13]: # Load the data, dataset (3) - AGLJ_J
columns = ['RIC', 'DateTimeL', 'Type', 'Price', 'Volume', 'L1 Bid', 'L1 Ask', 'Trade Sign']
aglj = pd.read_csv('AGLJ_J_04-Jan-2016_TO_10-May-2016_5days.csv', \
                  names = columns, skiprows = 1) # Load the data
aglj.sort_values(by = 'DateTimeL', inplace = True)
aglj.head(5)
```

```
Out[13]:
```

	RIC	DateTimeL	Type	Price	Volume	L1 Bid	L1 Ask	Trade Sign
1	AGLJ.J	736333.382013	Quote	0.0	0.0	0.0	6800.0	0
0	AGLJ.J	736333.382013	Trade	6750.0	366.0	0.0	0.0	0
2	AGLJ.J	736333.382015	Trade	6750.0	374.0	0.0	0.0	0
4	AGLJ.J	736333.382015	Quote	0.0	0.0	6735.0	6800.0	0
3	AGLJ.J	736333.382015	Quote	0.0	0.0	0.0	6800.0	0

The first thing to notice is the `DateTimeL` column. This is another format you may encounter when using tick data, and it represents a *serial date* number expressing dates and times. Specifically, it counts the number of days from a given date, which is called `base_date`. In our case, the `base_date` is January 1, 0000. For example, the 736333.382013 translates to the 736333rd day since January 1, 0000. The decimal part, represents the fraction of the day. In our example the 736333rd day is January 4, 2016, and the fraction 0.382013 corresponds to 09:10:05.

With the help of the `datetime` Python library, we will transform the serial date to a human-readable one. But this is not without problems! The `datetime` library calendar begins from Jan 1, 0001, as historically, there is no year 0000. On the other hand, indexing from 0 is preferred for scientific calculations. Students need not go deep into this matter, but this is a good example of the different notations that one faces when dealing with tick data.

```
In [14]: from datetime import datetime, timedelta

def convert_serial_to_datetime(serial):
    # Define the base date as January 1, 0001
    base_date = datetime(1, 1, 1)

    # Split the serial date into days and fractional days
    days = int(serial)
    fractional_day = serial - days

    # Adjust for the lack of year 0 by subtracting 367 days
    adjusted_days = days - 367

    # Calculate the target date by adding the adjusted days and fractional day (con
    target_date = base_date + timedelta(days=adjusted_days) + timedelta(seconds=fra

    # Return the target date in human-readable format
    return target_date.strftime('%Y-%m-%d %H:%M:%S')

aglj['timestamp'] = aglj['DateTimeL'].apply(convert_serial_to_datetime) # Convert t
aglj['timestamp'] = pd.to_datetime(aglj['timestamp']) # Convert the timestamp to da
aglj.set_index('timestamp', inplace=True) # Set the timestamp as the index
aglj.head(5)
```

Out[14]:

	RIC	DateTimeL	Type	Price	Volume	L1 Bid	L1 Ask	Trade Sign
timestamp								
2016-01-04 09:10:05	AGLJ.J	736333.382013	Quote	0.0	0.0	0.0	6800.0	0
2016-01-04 09:10:05	AGLJ.J	736333.382013	Trade	6750.0	366.0	0.0	0.0	0
2016-01-04 09:10:06	AGLJ.J	736333.382015	Trade	6750.0	374.0	0.0	0.0	0
2016-01-04 09:10:06	AGLJ.J	736333.382015	Quote	0.0	0.0	6735.0	6800.0	0
2016-01-04 09:10:06	AGLJ.J	736333.382015	Quote	0.0	0.0	0.0	6800.0	0

Let's now apply some cleaning filters:

Before we can effectively analyze high-frequency financial data, it's crucial to clean and preprocess the dataset. This cleaning process helps to remove erroneous entries and adjust for certain market microstructure effects, ensuring that our subsequent analyses are based on accurate and meaningful data.

First, we remove entries with zero or negative prices, quotes, or volumes. These are likely data errors, as prices and volumes in financial markets should always be positive. Negative spreads (where the bid price exceeds the ask price) are also eliminated as they represent impossible market conditions under normal circumstances.

Next, we address the issue of split transactions. In high-frequency trading, a single large order may be executed as multiple smaller trades at the same timestamp. This can lead to an overrepresentation of certain price levels and potentially skew our analysis. To mitigate this, we aggregate trades occurring at the same timestamp, summing their volumes and calculating a volume-weighted average price (VWAP). This approach preserves overall trading information while avoiding the artificial inflation of trade counts. Similarly, we deal with intra-second quotes by keeping only the last quote for each timestamp. This is because, in a fast-moving market, the most recent quote is typically the most relevant for analysis.

```
In [15]: # Delete entries with a quote or transaction price equal to zero or being negative
prices = aglj[aglj['Type'] == 'Trade']['Price'] # Get the transaction prices
quotes = aglj[aglj['Type'] == 'Quote']['L1 Bid', 'L1 Ask'] # Get the quotes
print("Number of entries with a transaction price equal to zero or negative: ", (pr
print("Number of entries with negative or zero volume: ", (aglj[aglj['Type'] == 'T
print("Number of entries with a quote equal to zero or negative: ", (quotes <= 0).s

index_to_drop_quotes = quotes[quotes['L1 Ask'] <= 0].index.union(quotes[quotes['L1
aglj.drop(index=index_to_drop_quotes, inplace=True) # Drop the entries with zero or
```

```

# Delete entries with a negative spread
spreads = aglj[aglj['Type'] == 'Quote']['L1 Ask'] - aglj[aglj['Type'] == 'Quote']
index_to_drop_spreads = spreads[spreads < 0].index # Get the indices to drop
print("Number of entries with a negative spread: ", len(index_to_drop_spreads)) # C
aglj.drop(index=index_to_drop_spreads, inplace=True) # Drop the entries with negati

# Dealing with split transactions
dataset_on_trades = aglj[aglj['Type'] == 'Trade'] # Get the trades
unique_trades = pd.DataFrame()
unique_trades['DateTimeL'] = dataset_on_trades['DateTimeL'].groupby(dataset_on_trad
unique_trades['Volume'] = dataset_on_trades['Volume'].groupby(dataset_on_trades['Da
unique_trades['Price'] = (dataset_on_trades['Price'] * dataset_on_trades['Volume'])
unique_trades.drop(columns='DateTimeL', inplace=True) # Drop the DateTimeL column

# Dealing with intra-second quotes
dataset_on_quotes = aglj[aglj['Type'] == 'Quote'] # Get the quotes
unique_quotes = pd.DataFrame()
unique_quotes['DateTimeL'] = dataset_on_quotes['DateTimeL'].groupby(dataset_on_quot
unique_quotes['L1 Bid'] = dataset_on_quotes['L1 Bid'].groupby(dataset_on_quotes['Da
unique_quotes['L1 Ask'] = dataset_on_quotes['L1 Ask'].groupby(dataset_on_quotes['Da
unique_quotes.drop(columns='DateTimeL', inplace=True) # Drop the DateTimeL column

aggr_dataset_2 = pd.merge(unique_trades, unique_quotes, on='DateTimeL', how='outer'

aggr_dataset_2['Type'] = np.where(aggr_dataset_2['Price'].isnull(), 'Quote', 'Tra

aggr_dataset_2['Trade Sign'] = aglj[aglj['DateTimeL'].isin(aggr_dataset_2[aggr_data
index) & (aglj['Type'] == 'Trade')].groupby('DateT

aggr_dataset_2.reset_index(inplace=True) # Reset the index
aggr_dataset_2.index = pd.to_datetime(aggr_dataset_2['DateTimeL']).apply(convert_ser

aggr_dataset_2.head(5)

```

Number of entries with a transaction price equal to zero or negative: 0

Number of entries with negative or zero volume: 0

Number of entries with a quote equal to zero or negative: 17

Number of entries with a negative spread: 42

Out[15]:

	DateTimeL	Volume	Price	L1 Bid	L1 Ask	Type	Trade Sign
DateTimeL							
	2016-01-04 09:10:20	736333.382180	NaN	NaN	6751.0	6800.0	Quote NaN
	2016-01-04 09:10:20	736333.382182	NaN	NaN	6751.0	6800.0	Quote NaN
	2016-01-04 09:10:21	736333.382190	NaN	NaN	6755.0	6800.0	Quote NaN
	2016-01-04 09:10:21	736333.382192	NaN	NaN	6757.0	6800.0	Quote NaN
	2016-01-04 09:10:42	736333.382433	NaN	NaN	6759.0	6800.0	Quote NaN

In [16]:

```

# A significant decrease in the number of rows
print(f"The filtered dataset `aggr_dataset_2` has {len(aggr_dataset_2)} entries, wh

```

The filtered dataset `aggr\_dataset\_2` has 134724 entries, while the `aglj` dataset has 185210.

7. Order Flow Imbalance (OFI)

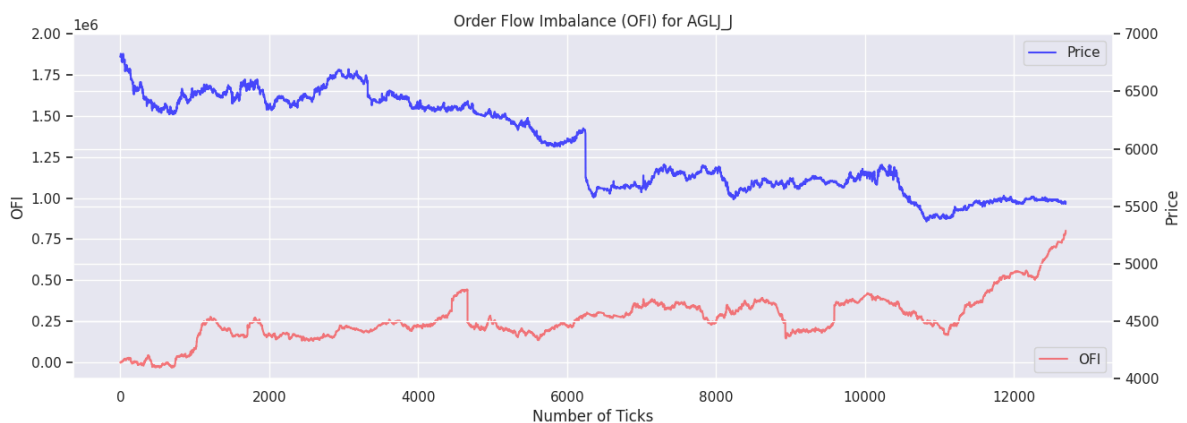
Order Flow Imbalance (OFI) is an important concept in high-frequency data analysis that provides insights into the buying and selling pressures in the market. It measures the net order flow by taking into account the direction and size of trades. We will calculate the simplest form of OFI by multiplying the `Trade Sign` column (a positive sign for buyer-initiated trades and a negative sign for seller-initiated trades) with the respective trade volumes. Over time, the cumulative sum of these signed volumes gives us the OFI. A positive OFI indicates net buying pressure, while a negative OFI suggests net selling pressure. By analyzing OFI, we can gain valuable insights into short-term price movements, liquidity dynamics, and potential future price trends (as theory suggests).

```
In [17]: # Figure 6
signed_volume = aggr_dataset_2[aggr_dataset_2['Type'] == 'Trade']['Volume'] * aggr_dataset_2['Trade Sign']
ofi = signed_volume.cumsum() # Calculate the cumulative sum of the OFI
prices = aggr_dataset_2[aggr_dataset_2['Type'] == 'Trade']['Price'] # Get the price

fig, ax = plt.subplots(figsize=(15, 5))
ax.plot(np.arange(len(ofi)), ofi, label = 'OFI', color = 'red', alpha = 0.5) # Plot OFI
ax.set_ylabel('OFI')
ax.set_xlabel('Number of Ticks')
ax.set_title('Order Flow Imbalance (OFI) for AGLJ_J')
ax.set_ylim(-100000, 2000000)
ax.legend(loc = 'lower right')

ax2 = ax.twinx()
ax2.plot(np.arange(len(prices)), prices, color='blue', label='Price', alpha=0.7)
ax2.set_ylabel('Price')
ax2.legend()
ax2.set_ylim(4000, 7000)
plt.show()

print("\n\nThe Pearson correlation between the price and OFI is: ", np.corrcoef(prices, ofi)[0][1])
```



The Pearson correlation between the price and OFI is: -0.6250454951480225

Exercise 5

Import the library `scipy` and compute the Spearman and Kendall correlation between the price and OFI. What do you see? Was that result what you expected?

The above result is somewhat counterintuitive and very common in practice. It highlights the importance of empirical analysis with real-world data rather than relying on only theoretical expectations.

As we can see, the price of AGLJ during these five days has a downward slope. On several occasions, the price has a short-term upward slope, but the trend in general is clear. Given the price, our expectation (based on theory and intuition) would be a downward slope for the OFI as well. But this is hardly the case since the OFI seems almost completely inconsistent with the price. The buying volume (as defined in this notebook) is rising which means that there are more buyers (takers) than sellers whereas, at the same time, the price is falling.

There could be several reasons for this phenomenon that could be very different if the data were retrieved from another exchange or for some other time interval:

- Market maker/Trader behavior: Market makers or large traders might be providing liquidity as prices fall, absorbing sell orders. This would show up as positive OFI (as they're buying) even as prices continue to decline.
- Large trades: A few large selling orders could outweigh the importance of many small buying orders.
- Market Microstructure Effects: In some market structures, large sell orders can trigger a series of smaller buy orders as they're filled, which could lead to this pattern.
- Time Frame Consideration: The relationship between OFI and price might be different over different time scales. What we're seeing could be a short-term effect that might reverse over longer periods.

Exercise 6

We deliberately gave a definition for buying and selling volume: Buying/Selling volume is the volume of the trades that were initiated by buyers/sellers (as takers). But now we must think of the buying and selling volume from the perspective of the trader's intention, which is the actual buying and selling volumes.

Hint: If a trader wants to buy, they can place a market order or a limit order.

Initiate a discussion in the forum with your ideas.

Exercise 7

We observed a counterintuitive result concerning the OFI of dataset (3): falling price with rising OFI. Now calculate the OFI for dataset (2) and plot it along with the price on the same graph (dual $\$y$ -axis). What do you see?

Let's now provide ideas for plotting the best bid and ask along with the price of AGLJ: we will use the step function for the quote levels and the scatter for the price.

```
In [18]: # Figure 7

zoomed_in_quotes_price = aggr_dataset_2.loc['2016-01-04'].iloc[15000:17000, :] # SL

fig, ax = plt.subplots(figsize=(15, 5))

ax.scatter(x = zoomed_in_quotes_price[(zoomed_in_quotes_price['Type'] == 'Trade')
    y = zoomed_in_quotes_price[(zoomed_in_quotes_price['Type'] == 'Trade')
    label='Buyer initiated Trades', alpha = 0.7, s = 40) # Plot the tra
ax.scatter(x = zoomed_in_quotes_price[(zoomed_in_quotes_price['Type'] == 'Trade')
    y = zoomed_in_quotes_price[(zoomed_in_quotes_price['Type'] == 'Trade')
    label='Seller initiated Trades', alpha = 0.7, s = 40) # Plot the tr
ax.step(zoomed_in_quotes_price[zoomed_in_quotes_price['Type'] == 'Quote'].index, \
    zoomed_in_quotes_price[zoomed_in_quotes_price['Type'] == 'Quote']
    label='L1 Bid', alpha = 0.8, where = 'post') # Plot the L1 Bid
ax.step(zoomed_in_quotes_price[zoomed_in_quotes_price['Type'] == 'Quote'].index, \
    zoomed_in_quotes_price[zoomed_in_quotes_price['Type'] == 'Quote']
    label='L1 Ask', alpha = 0.4, where = 'post') # Plot the L1 Ask

ax.set_title("AGLJ_J trades and L1 Bid/Ask")
ax.set_xlabel("Time")
ax.set_ylabel("Price")
ax.legend()
plt.show()
```



Exercise 8

In Figure 7, you can see the plot of the best quotes plotted along the price. Do the following:

- Be certain you can explain the argument `where = 'post'` inside the step plot function.

- By checking the `matplotlib` documentation, enhance the above graph by adjusting the size of the scatter marker using the size of the trade. So, if trade A is larger than trade B, then marker A should be larger than marker B. You can choose a different date and slice of the dataset. Paste your results in the forum.

8. Resampling

In the last section, we will be creating the time bars everyone is comfortable with. For this reason, we will use the `resample` method that works very well with a datetime index. We will create datasets for several timeframes and will showcase how some statistics change as the timeframe becomes larger. The dataset of our choice will be dataset (2): `rvn_usdt`.

```
In [19]: rvn_usdt.resample('1min', label = 'right').agg({'price': 'ohlc', 'qty': 'sum'}).head
```

```
Out[19]:
```

	price			qty	
	open	high	low	close	qty
timestamp					
2023-04-01 00:01:00	0.02588	0.02594	0.02588	0.02594	270588.0
2023-04-01 00:02:00	0.02594	0.02596	0.02594	0.02596	354425.3
2023-04-01 00:03:00	0.02597	0.02600	0.02596	0.02599	423137.4
2023-04-01 00:04:00	0.02598	0.02598	0.02592	0.02593	96082.3
2023-04-01 00:05:00	0.02594	0.02594	0.02594	0.02594	42425.7

Exercise 9

Explain the argument `label = right`. Discuss your ideas in the forum. Also, find a suitable library to plot the 'candles' along with the volume.

```
In [20]: #Create the resample function
def resample(df: pd.DataFrame, freq: str) -> pd.DataFrame:
    """
    Resample the data to a given frequency.
    """
    resampled_df = df.resample(freq, label = 'right').agg({'price': 'ohlc', 'qty': 'sum'})
    resampled_df.columns = resampled_df.columns.droplevel(0) # Drop the first level
    return resampled_df

timeframes = ['1min', '2min', '5min', '10min', '15min', '30min', '1h', '2h', '4h',
               '1d', '5d', '1w', '1mo', '3mo', '6mo', '1y']

statistics = {} # Create a dictionary to store the statistics
for timeframe in timeframes:
    temp = resample(rvn_usdt, timeframe)['close'] # Resample the data
    temp_returns = np.log(temp / temp.shift(1)).dropna() # Calculate the Log returns
    statistics[timeframe] = {'mean': temp_returns.mean(), 'std': temp_returns.std()}
```

```
In [21]: #Figure 8
pd.DataFrame(statistics).T.plot(subplots = True, figsize = (15, 5), layout = (2, 2))
plt.show()
```



The above results are somehow expected and show us the properties that we lose or gain by aggregating the high-frequency data.

- The mean return is close to 0 for small timeframes. Since each trade corresponds to small price increments, it is natural that the price changes in small timeframes were not able to accumulate and show a mean different than zero.
 1. Microstructure Noise: At very high frequencies, returns are heavily influenced by market microstructure effects such as bid-ask bounce, which can mask any true price trend over short intervals.
 2. Martingale Property: In a fair market, price processes are often modeled as martingales, where the expected future price given all past information is the current price. This implies that expected short-term returns should be zero.
- Standard Deviation: The increase in standard deviation with larger timeframes is consistent with the properties of Brownian motion (more in the Derivative Pricing course), which are used to model price processes. Under the assumptions of Brownian Motion, variance scales linearly with time, so standard deviation scales with the square root of time.
- The slight upward slope in skewness as the timeframe gets larger could indicate that the positive price movements tend to be larger than the negative ones over longer periods.
- More importantly, the high kurtosis at small timeframes reflects the leptokurtic nature of high-frequency returns. More specifically:
 1. Fat Tails: The extreme events are common in leptokurtic distribution, more common in relation to the normal distribution.

2. As we aggregate to the larger timeframes, the central limit theorem takes effect and the return distribution approaches normality. However, in larger timeframes, the returns still do not reach normality.

Exercise 10

Create time bars for the `rvn_usdt` pair as above but now include two more columns:

- A column named `buying_volume` that will consist of the aggregated volume of the trades attributed to the `is_buyer_maker = False`
- A column named `selling_volume` that will consist of the aggregated volume of the trades attributed to the `is_buyer_maker = True`

Exercise 11

In explaining the statistics of Figure 8, we used the central limit theorem to argue for the convergence of the returns distribution with the normal distribution. But we did not actually aggregate the returns. Rather, we used the `close` column of each timeframe to calculate the kurtosis. Is that argument used correctly or not? Why? Do the same as above but instead of the `close` price of each timeframe, calculate the `sum()` of the log returns. Does this change anything?

9. Conclusion

This lesson has provided an introduction to tick data, its characteristics, and its importance in financial markets. We've explored the structure of tick-by-tick trade data and Level 1 data, demonstrating how to handle, clean, and plot these high-frequency datasets.

Key takeaways include:

1. The granularity of tick data provides insights into market microstructure that are not visible in lower-frequency data.
2. Cleaning is an essential preprocess step that should include the handling of split transactions and intra-second quotes (among others).
3. Order flow imbalance (OFI) can provide valuable insights into short-term market dynamics, but its interpretation requires careful consideration of market context and can be misleading.
4. Resampling tick data to various timeframes reveals how statistical properties of returns change with time aggregation, highlighting the trade-offs between granularity and noise.
5. Working with tick data presents unique challenges, including data volume, cleaning requirements, and the need for specialized analytical techniques.

References

- Gebbie, Tim, and Edward Nonyane. "TRTH JSE AGLJJ Intraday Transaction Test Data." Mendeley Data, V2, 2019.
- Harris, Larry. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2003.
- Hautsch, Nikolaus. *Econometrics of Financial High-Frequency Data*. Springer Science & Business Media, 2011.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Murphy, Kevin Patrick. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.

- **Required pages:** Sections 7.5.1 Basic - 7.5.2 Connection Between SVD and EVD, pp. 259-261 under Section 7.5 SVD <https://probml.github.io/pml-book/book1.html>
- **Estimated reading time:** 20 minutes

Singular Value Decomposition of Matrices

|||:--:--| **Reading Time** 60 minutes || **Prior Knowledge** Basic Matrix Operation, Linear Algebra, Basic Python || **Keywords** Singular Value Decomposition (SVD), Full and economy SVD, matrix approximation |

In the previous lesson, we introduced methods to decompose a symmetric matrix. However, manytimes, the dataset or a matrix we have is not a symmetric matrix. In this lesson, we will introduce a general method to decompose a matrix. This method is called Singular Value Decomposition or SVD. We will go through the definition and properties of SVD and then present an application.

1. Introducing Singular Value Decomposition (SVD)

Singular value decomposition (SVD) is a very popular method in numerical linear algebra. It is commonly used to solve for a linear regression problem when the matrix is not squared. It can also be used as a data dimension reduction method. One of the examples is to use SVD as the basis for principal component analysis. SVD is also a foundational numeric method for machine learning algorithms.

In the last module, we discussed how to diagonalize a symmetric matrix. If A is a 3 by 3 symmetric matrix, we can diagonalize or factor A as follows:

$$A = \begin{bmatrix} | & | & | \\ v_1 & v_2 & v_3 \\ | & | & | \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{bmatrix} \begin{bmatrix} | & | & | \\ v_1^T & v_2^T & v_3^T \\ | & | & | \end{bmatrix}$$

where

v_1, v_2, v_3 are eigenvectors of A

$\lambda_1, \lambda_2, \lambda_3$ are eigenvalues of A

However, most of the time, the datasets or a matrix we have will not have symmetry. Hence, we need a method to diagonalize a general matrix. This method is singular value decomposition.

2. Definition of Singular Value Decomposition

2.1. Matrix Representation of SVD

Singular value decomposition is a method to decompose a matrix into three matrices. For any $m \times n$ matrix A with real entries, there exists a factorization of the form:

$$A = U \Sigma V^T$$

Where:

U is an $m \times m$ unitary and orthogonal, hence orthonormal matrix

Σ is an $m \times n$ rectangular diagonal matrix with non-zero real numbers on the diagonal

V^T is the transpose of V ; V is an $n \times n$ unitary and orthogonal (orthonormal) matrix

We can also rewrite the above formula in the following matrix form assuming $m \geq n$.

$$\begin{bmatrix} | & | & & | \\ a_1 & a_2 & \dots & a_n \\ | & | & & | \end{bmatrix} = \begin{bmatrix} | & | & & | \\ u_1 & u_2 & \dots & u_m \\ | & | & & | \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix} \begin{bmatrix} | & | & & | \\ v_1 & v_2 & \dots & v_n \\ | & | & & | \end{bmatrix}^T$$

where a are n column vectors in matrix A , u are m column vectors in matrix U , and v are n column vectors in matrix V .

The above matrix dimensions are as follows: $[m \times n] = [m \times m] [m \times n] [n \times n]$

The diagonal entries σ_i of Σ are known as the **singular values** of A .

They are typically arranged in descending order ($\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$). Since the above Σ matrix assumes $m \geq n$, there can be at most n positive singular values. The values of the rest of the rows in Σ are 0.

The vectors in matrix U are called the **left singular vectors** of A .

The vectors in matrix V are called the **right singular vectors** of A .

2.2. Geometric Interpretation of Singular Value Decomposition (SVD)

SVD can be interpreted geometrically as a composition of three transformations:

1. A rotation or reflection U
2. A scaling or dilating along coordinate axes Σ
3. Another rotation or reflection V^T

This interpretation helps in understanding how SVD captures the essential structure of linear transformations. The following graph illustrates the decomposition.

Figure 1: Visualization for Singular Value Decomposition

Diagram illustrating the Singular Value Decomposition (SVD) process, showing transformations: a linear transformation (A), rotations (U and V^T), and dilation (Σ), represented on a 2D plane with vectors.

3. Properties of Singular Value Decomposition (SVD)

In this section, we will discuss some of the key properties for SVD.

Property 1.

Since the vectors in U and V are unitary and orthogonal,

$$U^T U = U U^T = I_{m \times m} \text{ (identity matrix)}$$

$$V^T V = V V^T = I_{n \times n} \text{ (identity matrix)}$$

Property 2.

The number of non-zero singular values in matrix Σ is the rank of the matrix A .

Property 3.

Since singular values σ are arranged in descending order in the Σ matrix, the first column of U (which is u_1) and the first column of V (which is v_1) corresponding to σ_1 are more important in describing information in matrix A than the second columns of the U and V matrices. The first columns of U and V are also more important than the third columns in describing matrix A , and so on and so forth. Because of this property, when some σ 's values are very small, it means they contain only a very small amount of the information of matrix A . We can drop these σ s and their corresponding columns in the U and V matrices.

Property 4.

In the last section, we mentioned that the number of non-zero singular values in the Σ matrix is the minimum of m and n . Hence, there can be rows of all 0 s at the lower part of matrix Σ . As a result, when we conduct a dot product of matrix U with matrix Σ , the corresponding vectors of U will multiply with all the rows of 0 s in matrix Σ and give 0 result. With this result, we can simply SVD to a reduced form by dropping rows of all 0 in matrix Σ and the corresponding vectors in matrix U . The following Figure 2 demonstrates this property.

Figure 2: Full SVD vs. Economy/Reduced SVD

 Diagram comparing Full and Reduced Singular Value Decomposition (SVD), showing matrices A, U, Σ, and V^T with their respective components and dimensions.

In the first equation in Figure 2, we can see when we conduct a dot product for U and Σ , $\hat{U}^{\text{bot}}$ part of U will multiply with 0 of Σ . The result will be 0 . Hence, we can drop them without losing information in matrix A. The second equation is the reduced form of SVD. It is called **reduced SVD** or **economy SVD**.

4. Matrix Approximation Using SVD

In this section, we are going to demonstrate how to use SVD to approximate a matrix. This is the most popular reason to use SVD.

Assume for the $m \times n$ matrix A, the rank is r , and $r \leq \min(m, n)$. We can write matrix A's SVD as follows.

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} = \begin{bmatrix} u_{11} & u_{12} & \dots & u_{1m} \\ u_{21} & u_{22} & \dots & u_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ u_{r1} & u_{r2} & \dots & u_{rm} \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_r \\ 0 & 0 & \dots & 0 \end{bmatrix} \begin{bmatrix} v_{11} & v_{12} & \dots & v_{1n} \\ v_{21} & v_{22} & \dots & v_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ v_{r1} & v_{r2} & \dots & v_{rn} \end{bmatrix}^T$$

From the singular values in Σ , we know $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq \sigma_{r+1} = \dots = \sigma_{\min(m, n)} = 0$. We can expand matrix A as follows.

$$A = \sum_{k=1}^r \sigma_k u_k v_k^T = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T$$

This is called the Eckart-Young theorem. According to the theorem, we can represent matrix A by only using leading r vectors in matrix U and leading r vectors in matrix V because in the Σ matrix, there are only r non-zero σ s.

We also know that σ s in Σ are in descending order. If the σ s on the right side of the equation are too small, we can drop them from the equation. Then, we can approximate matrix A by using only σ s with large values. Say we only want to keep $\hat{r}$ leading σ s in the equation, where $r \geq \hat{r}$. The approximated matrix $\tilde{A}$ can be expressed as follows.

$$\tilde{A} = \sum_{k=1}^{\hat{r}} \sigma_k u_k v_k^T = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_{\hat{r}} u_{\hat{r}} v_{\hat{r}}^T$$

$\tilde{A}$ is the approximation of A . Based on the above equation and explanation, $\tilde{A}$ contains fewer data points than A but keeps the majority of the information from the original matrix A . We reduce data dimension using SVD to approximate a matrix. This is due to the property of SVD that allows us to solve least-squared issues or run machine learning algorithms more efficiently.

5. Application of Singular Value Decomposition (SVD)

In this section, we are going to use some financial data to demonstrate the usage of SVD. We will use the stock price data for Apple, Ford Motor, Pfizer, and Walmart to show how to apply SVD. First, let's download some Python libraries for this application.

```
In [ ]: !pip install pip --upgrade
        !pip install yfinance --upgrade
```

```
In [1]: import datetime
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import yfinance as yfin
import seaborn as sns
import math
from numpy import linalg as LA
```

5.1 Importing Data

Next, we are going to download stock price data from Yahoo Finance.

```
In [2]: # Download stock prices from Yahoo Finance and set the time period for download
start = datetime.date(2018, 1, 2)
end = datetime.date(2023, 12, 31)
stocks = yfin.download(["AAPL", "F", "WMT", "PFE"], start, end, auto_adjust = False)
stocks.head()
```

```
[*****100%*****] 4 of 4 completed
```

```
4 Failed downloads:
```

```
['PFE', 'AAPL', 'F', 'WMT']: YFRateLimitError('Too Many Requests. Rate limited. Try after a while.')
```

```
Out[2]: Ticker AAPL F PFE WMT
        Date
```

```
In [3]: stocks.index = pd.to_datetime(stocks.index).strftime("%Y-%m-%d")
```

5.2 Calculating Returns

Now, let's calculate the returns of these stock prices.

```
In [4]: # In order to calculate the returns of the stocks, we need to drop the NA rows.
stocks_returns = stocks[["AAPL", "F", "WMT", "PFE"]].dropna().pct_change()
stocks_returns = stocks_returns.dropna()
stocks_returns.head()
```

```
Out[4]: Ticker  AAPL  F  WMT  PFE
        Date
```

5.3 Computing the SVD

We now have the stock price return dataset for four stocks. We can simply calculate SVD using the following Python function.

```
In [5]: # Perform SVD for stock returns
U, s, VT = np.linalg.svd(stocks_returns)
```

```
In [6]: # Present the result
print("Stock Returns Matrix Dimension:")
print(stocks_returns.shape)
print("\nDimension of Matrix U:")
print(U.shape)
print("\nSingular values:")
print(s)
print("\nDimension of Matrix V^T:")
print(VT.shape)
```

```
Stock Returns Matrix Dimension:
(0, 4)
```

```
Dimension of Matrix U:
(0, 0)
```

```
Singular values:
[]
```

```
Dimension of Matrix V^T:
(4, 4)
```

6. Comparing PCA and SVD

So far, it has been very straightforward to calculate the SVD of a matrix using Python. Let's make things a little bit more interesting. We are going to show the connection between SVD and principal component analysis (PCA) (Brunton et al.).

In Module 1, we briefly discussed how to obtain principal components using eigenvectors and eigenvalues. In this section, we will show you how to use the SVD of a matrix to get

eigenvalues and eigenvectors.

Remember in Module 2, before we get to principal components, we need to standardize the data. Let's do that first.

```
In [7]: # Standardize stock returns dataset
stocks_returns_means = stocks_returns.mean()
stocks_returns_stds = stocks_returns.std()
standardized_returns = (stocks_returns - stocks_returns_means) / stocks_returns_std
standardized_returns.head()
```

```
Out[7]: Ticker AAPL F WMT PFE
Date
```

Next, we want to calculate the covariance of the standardized returns matrix. The matrix form of the covariance equation is as follows.

$$\frac{1}{\sqrt{n-1}} \sum_{i=1}^n \left(\text{standardized_returns}_{i,j} - \text{stocks_returns_means}_j \right) \left(\text{standardized_returns}_{i,k} - \text{stocks_returns_means}_k \right)$$

```
In [8]: # Calculate covariance for standardized return matrix
standardized_returns_dvd_sqrt_n = (standardized_returns / math.sqrt(len(standardized_returns) - 1))
standardized_returns_cov = standardized_returns_dvd_sqrt_n.T @ standardized_returns_dvd_sqrt_n
standardized_returns_cov
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[8], line 2
      1 # Calculate covariance for standardized return matrix
----> 2 standardized_returns_dvd_sqrt_n = (standardized_returns / math.sqrt(len(standard
      3 ized_returns) - 1))
      3 standardized_returns_cov = standardized_returns_dvd_sqrt_n.T @ standardized_re
      4 turns_dvd_sqrt_n
      4 standardized_returns_cov

ValueError: math domain error
```

Now, let's do some simple matrix manipulation of the covariance of standardized returns.

Let's assume matrix B is the $\frac{1}{\sqrt{n-1}} \sum_{i=1}^n \left(\text{standardized_returns}_{i,j} - \text{stocks_returns_means}_j \right) \left(\text{standardized_returns}_{i,k} - \text{stocks_returns_means}_k \right)$. Hence, we can rewrite the covariance of standardized returns as follows.

$$\text{Covariance} = B^T B$$

With SVD, we can write matrix B as $B = U \Sigma V^T$. Then, its transpose is $B^T = V \Sigma U^T$. Then, we can calculate the covariance of the standardized returns as follows.

$$\text{Covariance} = B^T B = V \Sigma U^T U \Sigma V^T = V \Sigma \Sigma V^T = V \Sigma^2 V^T$$

$$V^T$$

Because the property of $U^T U = I$ is an identity matrix, we can drop $U^T U$. Next, let's conduct the following calculation.

$$B^T B V = V \Sigma^2 \quad V^T V = \Sigma^2$$

We can drop $V^T V$ because $V^T V = I$ is an identity matrix. Does the equation above remind you of something? Yes, the equation shows that the column vectors in matrix V are the eigenvectors of $B^T B$. The squared singular values in matrix Σ are eigenvalues of $B^T B$. Hence, Bv_1 is the first principal component and the corresponding σ_1^2 is the variance of the dataset B explained by the first principal component.

Now, you actually have two ways to calculate principal components. The first method, as described in Module 1, Lesson 4, is by calculating the eigenvalues and eigenvectors of the covariance matrix of the data. The second method is to calculate the SVD of the data. Then, matrix V will contain eigenvectors, and squared singular values will be eigenvalues.

Now let's try both methods to get eigenvalues and eigenvectors.

```
In [ ]: # Use SVD to calculate eigenvectors and eigenvalues of the covariance matrix of sta
U_st_return, s_st_return, VT_st_return = np.linalg.svd(standardized_returns_dvd_sqr
print("\nSquared Singular values (eigenvalues):")
print(s_st_return**2)
print("\nMatrix V (eigenvectors)")
print(VT_st_return.T)
```

```
In [ ]: # Use the method from Module 1 Lesson 4 to calculate eigenvectors and eigenvalues o
eigenvalues, eigenvectors = LA.eig(standardized_returns_cov)
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
eigenvalues
```

```
In [ ]: eigenvectors
```

We can see from the above result that both methods generate the same eigenvalues. Both methods also generate the same eigenvectors except the first one. Looking closely, we can see they only differ in regards to the negative sign. For eigenvectors, they are actually the same because the negative sign does not change the direction nor the scale of both eigenvectors.

You can use either method to obtain the eigenvectors and eigenvalues of a covariance matrix. There is no agreement on which one is better. We usually use SVD for implementing PCA, image compression, and recommendation systems. We would use the method in Module 1, Lesson 4 for solving least-square problems.

One key difference between PCA and SVD is how they handle starting points. In general, PCA works on the covariance or correlation matrix. PCA also standardizes the data matrix.

However, SVD can simply work straight on the data matrix. If SVD does not standardize the data matrix, the resulting eigenvalues can differ from the eigenvalues calculated from PCA.

7. Conclusion

In this lesson, we first introduced SVD. We gave its definition, demonstrated its geometric interpretation, and discussed its properties. We then showed its application in Python. We showed how to use SVD to calculate eigenvalues and eigenvectors of a covariance matrix. We also pointed out the connections between SVD and PCA.

References

- Brunton, Steven L., et al. "Chapter 1: Singular Value Decomposition (SVD) and Principal Components Analysis (PCA)." University of Washington, 2015, <https://faculty.washington.edu/sbrunton/me565/pdf/CHAPTER1.pdf>.